

MEMENTO: A novel approach for class incremental learning of encrypted traffic

Francesco Cerasuolo^{a,*}, Alfredo Nascita^a, Giampaolo Bovenzi^a, Giuseppe Aceto^a,
Domenico Ciunzo^a, Antonio Pescapè^a, Dario Rossi^b

^a University of Napoli “Federico II”, Italy

^b Huawei Technologies France SASU, France

ARTICLE INFO

Keywords:

Traffic classification
Class incremental learning
Mobile apps
Encrypted traffic
Deep learning

ABSTRACT

In the ever-changing digital environment, ensuring the ongoing effectiveness of traffic analysis and security measures is crucial. Therefore, Class Incremental Learning (CIL) in encrypted Traffic Classification (TC) is essential for adapting to evolving network behaviors and the rapid development of new applications. However, the application of CIL techniques in the TC domain is not straightforward, usually leading to unsatisfactory performance figures. Specifically, the improvement goal is to reduce forgetting on old apps and increase the capacity in learning new ones, in order to improve overall classification performance—reducing the drop from a model “trained-from-scratch”.

The contribution of this work is the design of a novel fine-tuning approach called MEMENTO, which is obtained through the careful design of different building blocks: *memory management*, *model training*, and *rectification strategies*. In detail, we propose the application of *traffic biflows augmentation strategies* to better capitalize on old apps biflows, we introduce *improvements in the distillation stage*, and we design a *general rectification strategy* that includes several existing proposals.

To assess our proposal, we leverage two publicly-available encrypted network traffic datasets, i.e., MIRAGE19 and CESNET-TLS22. As a result, on both datasets MEMENTO achieves a significant improvement in classifying new apps (w.r.t. the best-performing alternative, i.e., BiC) while maintaining stable performance on old ones. Equally important, MEMENTO achieves satisfactory overall TC performance, filling the gap toward a trained-from-scratch model and offering a considerable gain in terms of time (up to 10× speed-up) to obtain up-to-date and running classifiers. The experimental evaluation relies on a comprehensive performance evaluation workbench for CIL proposals, which is based on a wider set of metrics (as opposed to the existing literature in TC).

1. Introduction

Traffic Classification (TC) plays a crucial role in various network management tasks, cyber-security, traffic differentiation, and resource allocation. The ability to accurately and efficiently classify network traffic objects into different application or service classes enables network operators to make informed decisions, enforce policies, and ensure optimal network performance [1,2].

According to the Ericsson Mobility Report of June 2023 [3], total mobile network traffic reached 126 EB per month in Q1 2023 and it is expected to achieve an impressive amount of 453 EB per month by the end of 2028. Therefore, it is necessary to effectively capitalize on this large amount of data.

Consequently, over the years both Machine Learning (ML) and Deep Learning (DL) approaches have been extensively explored for

network TC with the aim of making the process automatic and more effective [4–9]. In general, traditional ML- and DL-based approaches for network TC typically rely on *static* models that are trained on predefined classes and fixed feature sets. However, with the continuous evolution of network applications and the emergence of new traffic patterns, static models struggle to *adapt* and efficiently incorporate new classes without requiring from-scratch retraining on all data intended to be classified. This aspect is even more significant for the traffic generated by mobile devices, which is influenced by the release of new apps and the continuous update of those already released [10].

To overcome the limitations of static models, the research community is showing a growing interest in the application of *Class Incremental Learning* (CIL) techniques to network TC [11–20]. In detail, CIL offers

* Corresponding author.

E-mail addresses: francesco.cerasuolo@unina.it (F. Cerasuolo), alfredo.nascita@unina.it (A. Nascita), giampaolo.bovenzi@unina.it (G. Bovenzi), giuseppe.aceto@unina.it (G. Aceto), domenico.ciunzo@unina.it (D. Ciunzo), pescapè@unina.it (A. Pescapè), dario.rossi@huawei.com (D. Rossi).

<https://doi.org/10.1016/j.comnet.2024.110374>

Received 3 October 2023; Received in revised form 22 March 2024; Accepted 25 March 2024

Available online 28 March 2024

1389-1286/© 2024 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

a promising avenue for enhancing the adaptability, accuracy, and efficiency of traffic classifiers in dynamic network environments [21]. This learning paradigm offers a promising solution to the challenges posed by the evolving network traffic, as it facilitates efficient model updates and ensures accurate classification performance as new applications are released over time. Nevertheless, applying CIL to TC is a complex and multifaceted task, with previous studies emphasizing the necessity of additional research efforts within the networking community [20].

Accordingly, this work proposes MEMENTO, a *novel fine-tuning approach* aiming at solving the CIL task in TC and improving the performance of state-of-the-art (SOTA) approaches in such a challenging scenario. We delve into the detailed design choices of the three main building blocks of fine-tuning approaches for CIL, namely: *memory management*, *training procedure*, and *model rectification*.

In particular, the **technical innovations** of our proposal can be summarized as follows. First, we introduce *traffic biflows augmentation primitives* designed to generate synthetic biflows, aiming to address the significant imbalance between old and new applications. Secondly, we enhance the training procedure by incorporating a *balanced validation set* and modifying the distillation stage. Specifically, we propose a *selective smoothing* procedure to effectively preserve old knowledge. Thirdly, we present a *comprehensive definition for the correction layer* and assess various instances to mitigate the bias toward the most recently added applications. We leverage two publicly-available datasets MIRAGE19 [8] and CESNET-TLS22 [22] to evaluate our proposal. First, we assess our CIL solution in a small-increment scenario (39+1 apps) which represents the most common scenario for TC in which a *classifier is required to learn a newly released app*. Secondly, we assess MEMENTO performance in a big-increment scenario (20+20 apps) which stands for the *customization of a pre-trained classifier with a defined set of apps*. Both scenarios are considered for the two datasets (in the latter case by retaining the top-40 apps of CESNET-TLS22). Furthermore, we validate our proposal on a *larger-scale TC task*. In detail, we consider a wider set of apps (i.e., the most representative 80 from CESNET-TLS22), evaluating both small-increment (79+1 apps) and big-increment (40+40 apps) scenarios. Then, we assess the time efficiency gained by employing the proposed incremental approach (MEMENTO) in contrast to performing a train-from-scratch. Finally, considering the 80 most representative apps of CESNET-TLS22, we evaluate MEMENTO in a multi-increment scenario (20+20+20 apps).

The rest of the paper is organized as follows. Section 2 provides the necessary background for CIL and its formal definition (Section 2.1) and reviews the TC literature dealing with the CIL issue (Section 2.2). Then, Section 3 introduces our proposal and all its building blocks; Section 4 details the experimental setup considered for the evaluation; Section 5 reports, for each building block, the experimental results pertaining to the analysis of different design choices; Section 6 presents our proposal MEMENTO and evaluates it with MIRAGE19 and CESNET-TLS22 in different scenarios. Section 7 ends with conclusions and future directions.

2. Background and related works

In this section, we provide a brief refresher on the fundamental concepts and terminology of CIL (Section 2.1). Then, we delve into the exploration of related works for CIL in the TC domain (Section 2.2).

2.1. Preliminaries on class incremental learning

CIL concerns how to include knowledge of new classes (viz., apps) in the learning process – retaining the already acquired knowledge about old classes – without performing a retraining-from-scratch. The purpose of model update procedures is to mitigate the problem of *catastrophic forgetting*, which refers to the model's tendency to overwrite previously acquired knowledge with new information [23], and *intransigence*, that is the model's inability to learn new apps.

Before illustrating the possible strategies to address these principles, we first introduce the necessary symbolism pertaining to DL approaches. Specifically, we consider a dataset $D = \{(\mathbf{x}_1, y_1) \dots (\mathbf{x}_N, y_N)\}$ consisting of N input samples, where each sample $\mathbf{x}_n \in \mathcal{X}$ (e.g., a biflow) is associated with a class label $y_n \in C = 1, 2, 3, \dots$ (e.g., a specific app/service). Therefore, the traffic classifier aims to train a set of parameters θ in order to learn a function $\gamma : \mathcal{X} \rightarrow C$, where $\gamma(\mathbf{x}) = \hat{y} \in C$ represents the predicted label for the input $\mathbf{x}$. In what follows, we denote by D_c the dataset that contains only biflows belonging to class c .

In general, **two** separate parts can be identified in DL models: a **backbone** (with corresponding set of parameters summarized as θ^{back}), which performs a *feature extraction* function $\phi : \mathcal{X} \rightarrow \mathcal{V}$ and transforms the input sample $\mathbf{x}$ into a *feature vector* $\mathbf{v}$ by projecting $\mathbf{x}$ into the *latent space* $\mathcal{V}$; a **head** (with corresponding set of parameters summarized as θ^{head}), which implements a *classification* function $h : \mathcal{V} \rightarrow \mathcal{Z}$ that transforms the feature vector into *logits* $h(\phi(\mathbf{x})) = \mathbf{o}(\mathbf{x}) \in \mathcal{O}$. Finally, logits are converted into class probabilities via the usual softmax activation, viz., $[softmax(\mathbf{x})]_c = p_c(\mathbf{x}; C) = e^{o_c(\mathbf{x})} / \sum_{j=1}^{|C|} e^{o_j(\mathbf{x})}$, with $p_i(\mathbf{x}; C)$ denoting the i th class probability. The predicted label is then obtained as $\hat{y} \triangleq \argmax_{c \in C} \{p_c(\mathbf{x}; C)\}$.

The CIL problem can then be formally defined as follows. At each episode m , a new set of apps C^{new} is introduced alongside a labeled dataset D^{new} . The objective is to **update the existing classifier** (which initially can only discriminate among the apps C^{old}) so as to incorporate the new ones (C^{new}) and be able to discriminate among the full set $C^{all} = C^{old} \cup C^{new}$. This incremental update employs (i) the existing model parameters θ^* and (ii) the new samples D^{new} . Additionally, the update may leverage (iii) a subset of biflows from the old apps as a memory, denoted as D^{mem} , to further mitigate catastrophic forgetting.

Each model update strategy falls into one of the following families: *Fine-Tuning* [11,12,14,15,17,18], which refines the whole model with new data; *Fixed-Representation* [24,25], which relies on a rich backbone and updates the sole model head; *Model-Growth* [13,16,19], which designs new models for new classes and makes predictions combining all models outputs. To counter the forgetting phenomenon – which rises in more challenging setups – sophisticated strategies have been proposed to mitigate forgetting, namely *rehearsal*, *regularization*, and *bias correction*.

2.2. State of the art of class incremental in TC

In this section, we discuss the related literature proposing CIL solutions for TC (Section 2.2.1), with a focus on works from the fine-tuning and model-growth families (resp. Sections 2.2.2 and 2.2.3). Finally, we position our work with respect to the literature discussed (Section 2.2.4).

2.2.1. At a glance

CIL is a topic of great interest widely explored in the Computer Vision (CV) field but still largely unexplored in TC.

Indeed, existing works addressing TC in the encrypted domain are based on the traditional paradigm of training a classifier *from scratch* [6,9,26–31], namely, they neglect the scenario in which new classes are introduced incrementally into the models. However, due to the dynamic nature of Internet traffic and the demand for rapidly-updated classifiers, the conventional procedure is not efficient, since it is highly resource-demanding and requires the entire dataset to train the model. Hence, the need for CIL in TC has become of primary importance. Also, with the increasing adoption of DL techniques for TC, the works in the research community explore how to enable DL models to continuously learn and discriminate new apps while keeping already acquired knowledge.

Table 1 reports SOTA works by considering different aspects, detailed in the following. As previously mentioned, the main **Families** of

Table 1

Related works proposing CIL approaches for network TC tasks and analyzing building blocks.

Year	Paper	CIL method	Family	Input data	Backbone	Head	Rehearsal	Bias Corr.	Distill.	Mem. Aug.
2020	Zhang et al. [11]	Fine-Tuning	FT	PB (as images)	1D-2D-3D-CNN	DE	○	○	○	○
2020	Sang et al. [12]	iCaRL	FT	PL, PB, CT	AIBMF	DE	●	○	●	○
2021	Chen et al. [13]	OvA-Ens	MG	PL	GRU ensemble	DE	●	○	○	○
2021	Bovenzi et al. [14]	iCaRL+	FT	PS, DIR, IAT, WIN	2D-CNN	DE	●	○	●	○
2022	Wu et al. [15]	BiC	FT	PL, IAT, BR	3D-CNN	DE	●	●	●	○
2023	Song et al. [16]	OCC-Ens	MG	PL, DIR, IAT, WIN, FLG, TTL	LSTM	–	○	○	○	○
2023	Ma et al. [17]	iCaRL+ *	FT	PB (as images)	ResNet18	PA	●	●	●	○
2023	Zhu et al. [18]	Fine-Tuning	FT	PB (as images)	ResNet18	DE	●	○	○	●
2023	Cai et al. [19]	Prompting	MG	PB	FS-Net	DE	●	○	○	○
	This paper	MEMENTO	FT	PL, DIR, IAT, WIN	2D-CNN	DE	●	●	●	●

CIL Method: One-vs-All Ensemble (OvA-Ens), One-Class-Classifer Ensemble (OCC-Ens), and iCaRL+ with pre-allocated head and modified loss (iCaRL+ *). **Input Data:** Payload Bytes (PB), Packet Length (PL), L4 Payload Size (PS), Content Type (CT), Byte Rate (BR), Direction (DIR), Inter-Arrival Time (IAT), TCP Windows Size (WIN), TCP Flags (FLG), Time-To-Live (TTL). **Family:** Model-Growth (MG), Fine-Tuning (FT). **Backbone:** Convolutional Neural Network (CNN), Gated Recurrent Unit (GRU), Long Short Term Memory (LSTM). **Head:** Dynamically-Expanded (DE), Pre-Allocated (PA). **Type of analysis:** ● in-detail, ● partial, ○ lacking.

CIL solutions are *fine-tuning*, *fixed-representation*, and *model-growth*. Surveying the TC literature proposing CIL solutions, the first outcome is the lack of proposals for the fixed-representation family: such approaches are often treated as baselines because of the poor plasticity leading to high intransigence. Regarding the other two families, fine-tuning approaches dominate, constituting 6 of the 9 proposals [11,12,14,15,17,18], while the model-growth family is represented by the remaining 3 works, i.e., [13,16,19].

Regarding the **Input Data**, it is worth emphasizing that [12] leverages multi-modal input exploiting three views relating to packet length, packet content type, and payload bytes. Conversely, the other SOTA approaches considered leverage different sets of features extracted from the header of packets [13–16], such as packet length, transport-layer payload size, direction, inter-arrival time, TCP windows size, TCP flags, time-to-live, and bytes rate, or payload bytes arranged as images [11,17,18].

Concerning the selected **Backbone**, most of considered works leverage CNN-based architectures [11,12,14,15,17,18] whereas the remaining ones [13,16,19] are based on RNNs.

Regarding the model **Head**, all considered approaches dynamically expand it during the incremental stage [11–15,18,19], except for Ma et al. [17], which employs a static head with a pre-allocated number of neurons. The latter approach allows for the inclusion of new apps until reaching the saturation of unassigned neurons, necessitating from-scratch retraining. Notably, [12] does not leverage the model head for classification but employs a Euclidean-based distance metric in the feature space. Additionally, in [16], the authors propose a classifier based on the reconstruction error obtained by each LSTM, one per application.

In examining mitigation strategies, it is evident that the TC domain has seen only a partial exploration of potential solutions. The predominant strategy is **Rehearsal**, employed by 7 out of 9 works, although all these works except [13] utilize only one procedure for sample selection in the rehearsal phase. **Distillation**, the sole regularization strategy in SOTA approaches (4 out of 9 works), introduces a distillation loss objective during model updates. Conversely, **Bias Correction** is only employed by a few works. Notably, a recent work [18] introduces the novel approach of enforcing **Memory Augmentation**.

In the following, we discuss papers on *fine-tuning* and *model-growth* approaches, as our literature review did not reveal any work based on fixed-representation techniques.

2.2.2. Discussion of fine-tuning techniques

Zhang et al. [11] is among the first to propose a CIL-powered DL classifier for TC, whose proposal presents a closed-loop TC system capable of extending its knowledge. However, the authors consider a small use case with 10 apps and adopt a simple fine-tuning strategy to introduce new apps focusing on a single model update.

Fine-tuning approaches commonly use rehearsal techniques [12,14,15,17,18] to refresh knowledge of *old* apps in the incremental training

stage by storing their biflows. To address the significant imbalance between new and old apps, data generation techniques have been proposed. For instance, Zhu et al. [18] employ Generative Adversarial Networks (GANs) to generate synthetic biflows, reducing the gap and enhancing the incremental training stage alongside stored biflows in memory.

To preserve further old knowledge, several approaches [12,14,15,17] incorporate Knowledge Distillation (KD) [32], minimizing changes in model weights that contain valuable information for the old apps. KD is integrated into the loss function, combined with other terms to guide incremental training.

Studies reveal that DL models often exhibit task-recency bias, and methods to address the imbalance toward new apps have been proposed [33,34]. Correction can be explicit, as seen in the addition of a component [15], or implicit, using a non-trainable correction [17]. In detail, Wu et al. [15] incorporate a trainable correction layer, adjusting new apps logits to achieve a balance. Conversely, Ma et al. [17] introduce a balance factor in the loss function, derived from normalizing a bias vector representing the model's bias toward the class.

It should be noted that existing fine-tuning approaches *have limitations related to (i) their constrained design* (which only partially encompasses the full range of design choices) and *(ii) their limited evaluation scenarios* (which rely on datasets with a few applications, resulting in TC tasks of impractical utility).

2.2.3. Discussion of model-growth techniques

Fine-tuning approaches are grouped via their mitigation strategies, while model-growth ones [13,16,19] differ in both the composite structure and the output combination strategy.

To address CIL, Chen et al. [13] propose an ensemble of binary classifiers using the One-vs-All strategy and rehearsal technique, referred here to as OvA-Ens. In detail, each model is trained with binary cross-entropy and employs sigmoid activation. The incremental phase includes selective retraining of old app classifiers, minimizing false positives for new app samples. However, this approach increases space and time complexity, requiring a model per app or $|C^{old}| + |C^{new}|$ models.

Regarding Song et al. [16], they propose to train a new LSTM model for each traffic type (viz. class) to extract fingerprints, allowing incremental extension by training additional parameters for new traffic types. Hereinafter, we refer to this approach as OCC-Ens (One-Class-Classifer Ensemble). It is worth emphasizing that OCC-Ens presents the same space and time complexity of OvA-Ens since it also requires a model per app.

Furthermore, Cai et al. [19] envision a new approach based on prompt learning to tackle the CIL problem in TC. This kind of approach, grouped into the model-growth family [35], has been proposed by Wang et al. [36] in CV and involves (i) a prompt expansion and (ii) an incremental fine-tuning of the model.

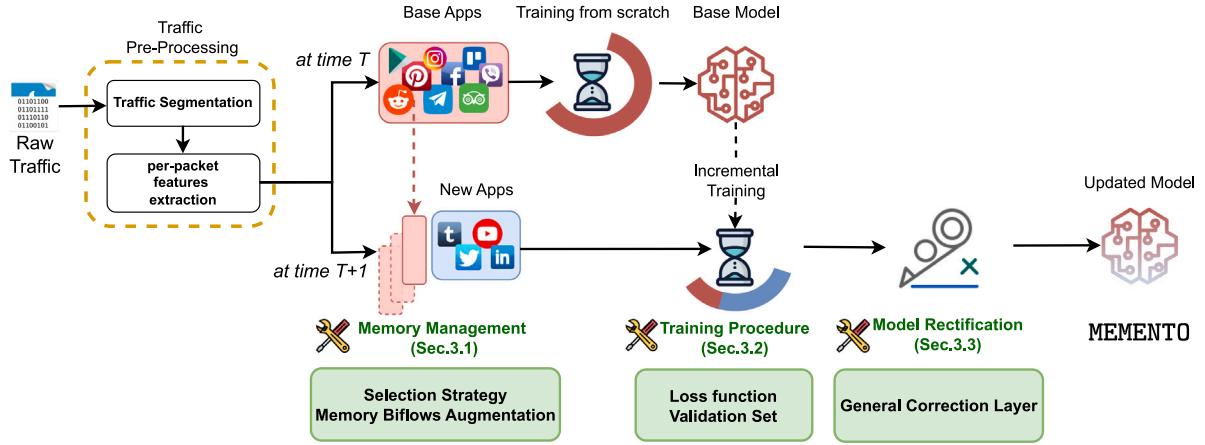


Fig. 1. Incremental Learning procedure leading to MEMENTO approach. The main investigated aspects are highlighted with a screwdriver-and-wrench icon () along with the section they are introduced in and the relative analyses.

Despite model growth approaches show satisfactory classification performance in incremental stages, *their limited scalability* represents an obstacle to their adoption in encrypted TC, due to the extensive number of apps and services *requiring numerous models to be handled*.

2.2.4. Positioning of our work

This work introduces MEMENTO, a novel fine-tuning approach whose high-level objectives are (a) to improve classification performance of new apps (i.e., combating intransigence), while (b) maintaining the performance of old apps stable (i.e., limiting catastrophic forgetting). Our solution is built upon the fine-tuning family (as model-growth techniques have scalability concerns with a large pool of apps) and bridges the gap of existing fine-tuning methods in TC. MEMENTO represents a significant leap in applying incremental learning techniques to real-world problems in TC.

This work comes in continuity with our two previous works [14,20], both relying on the MIRAGE19 dataset for the experimental evaluation. The first [14] delves into an in-depth study of the CIL solution iCaRL, resulting in an optimized TC-focused version named iCaRL+. The latter [20] is a recent benchmarking study that compares a wide variety (i.e., 16 different proposals) of SOTA solutions for CIL in CV, together with recent CIL proposals for TC (including iCaRL+).

Based on the knowledge and results gathered from the aforementioned works (and in contrast to their comparative focus), this paper *devises a novel proposal* which arises from a procedure that systematically analyzes and dissects the three building blocks that are required to incrementally update a DL model (belonging to the fine-tuning family): memory management, training procedure, and model rectification. This is accomplished by performing a *thorough exploration of several design options for each block*, underlining their pros and cons.

The effectiveness of MEMENTO is confirmed over a *relevant set of CIL SOTA approaches* (i.e., the bests observed in [20]) in a *wider range of incremental evaluation scenarios* (w.r.t. [14,20]). Equally important, this work *leverages both MIRAGE19 and CESNET-TLS22*, which are two recent public datasets that encompass encrypted traffic (as opposed to [14,20], using only the former). The first is composed of traffic generated by 40 Android mobile apps, while the second comprises 191 generic services captured in a network environment.

This comprehensive evaluation sets our work apart from others [12,13] relying on a single, private, and non-human-generated dataset, ensuring realism and reproducibility. Also, unlike works [11,15–19] leveraging also the outdated and class-limited ISCX VPN-nonVPN dataset [37] (collected in 2016 and consisting of up to 14 traffic classes), our evaluation explores realistic incremental learning scenarios with a larger set of base classes and bigger increments.

3. Methodology

In this work, we define a new CIL approach named MEMENTO: **distill augmented knowledge, smooth new information, and rectify both**. Our proposal relies on *three main aspects* to improve the model building: *memory management, training procedure, and model rectification*. These are visually positioned in the incremental learning procedure leading to MEMENTO in Fig. 1. Each aspect has a unique focus, enabling independent exploration for diverse contributions to the overall model performance. Additionally, within each aspect, there are optional processing steps and multiple alternative processing methods, which may either remain unused or diverge from the design choices found in the referenced SOTA.

We emphasize that, with respect to each block, we not only systematically investigate strategies already defined in the literature, but also *expand and generalize their formulations*. Furthermore, we *introduce new components* to achieve a more balanced integration of old and new knowledge, such as traffic augmentation primitives and the addition of a temperature scaling factor tailored for new classes.

Accordingly, Section 3.1 describes the memory management of MEMENTO, including the sample selection strategies and the augmentation technique. Then, Section 3.2 provides all the details regarding the training procedure, including the optimization routine and the definition of the CIL loss function. Finally, Section 3.3 introduces a general correction layer for the correction of residual bias toward new classes induced by the training phase.

3.1. Memory management

A key strategy to mitigate catastrophic forgetting involves utilizing a subset of old apps' biflows, referred to as “memory” (denoted as D^{mem}), during incremental training. The use of memory addresses the practical need to avoid storing the entire base dataset, allowing for a trade-off between the high cost of *training-from-scratch* and performance figures. In terms of memory, two design choices need consideration: (i) determining how the memory is built/updated and (ii) introducing augmentation procedures to reduce the imbalance in biflows between old and new apps.

Remarks on complexity. It is crucial to note that both design choices influence the *temporal complexity* of the resulting approach. Each technique for constructing memory impacts temporal complexity differently, based on the expected operations by various sampling techniques. Simultaneously, introducing samples generated with augmentation techniques directly affects training times by expanding the training set for incremental training (since it enlarges the equivalent memory).

3.1.1. Biflows selection strategies

Various strategies exist for selecting biflows from previous incremental steps' training data. Commonly employed in CV literature [38–40] are (i) random selection, (ii) herding selection, (iii) entropy-based selection, (iv) distance-based selection, and (v) rainbow memory. Regardless of the selection strategy, the procedure maintains a *stratified* approach by choosing the same number of biflows for each old app, ensuring consistency with respect to C^{old} . Choosing an equal number of biflows for memory, irrespective of class imbalance, is vital to retaining information about old apps without introducing sampling biases. Consequently, each procedure is applied independently to each app $c \in C^{old}$, allowing for *parallelization*, along with the associated dataset D_c^{old} .

- **Random:** this naïve strategy randomly selects training set biflows belonging to the generic old apps.
- **Herding [38]:** this method selects biflows based on the feature (viz., latent) space $\mathcal{V}$. Going into details, this strategy selects biflows with the shortest distance with respect to the class-average centroid in the latent space, i.e., $|D_c^{old}|^{-1} \sum_{x \in D_c^{old}} \mathbf{v}(x)$.
- **Entropy-based [39]:** The entropy of the output soft-max distribution, measuring confidence in classification, is calculated as $-\sum_{c \in C^{old}} p_c(x) \log p_c(x)$ for a given sample. Consequently, a higher entropy indicates a higher likelihood of selecting the biflow. Hence, this strategy favors biflows associated with less confident predictions, characterized by a more uniformly-distributed soft-max output.
- **Distance-based [39]:** assuming that the change in parameters is regularized, the feature space and the decision boundaries do not vary much. Hence, the biflows that lie close to the boundary would act as boundary-defining biflows. Accordingly, a pseudo-distance from the decision boundary, corresponding to $1/o_c(x)$ (i.e., the inverse of the logit of the generic class), is computed. Thus, less-distant biflows are preferred via this strategy.
- **Rainbow Memory [40]:** this method selects biflows based on their representation quality and diversity within the dataset. The selection involves perturbing each biflow using various transformation techniques, and an uncertainty score is computed by measuring the variation in model responses to the original and perturbed versions of the same biflow. A diversity-based ranking is then established and biflows are chosen at fixed intervals to ensure comprehensive coverage of the entire input data space.

3.1.2. Traffic biflows augmentation

Regardless of the selection strategy, the number of biflows that can be stored is usually limited. To overcome the scarcity of old apps biflows, *relevant transformations* can be applied to the memory biflows to obtain synthetic ones and further mitigate forgetting effects. In this work, these transformations are *applied to the inter-arrival time* (IAT) field. This field is arguably the most sensitive to the network conditions, therefore this choice has the additional benefit of improving generalization of the trained classifier. Specifically, the transformations are defined as follows:

- **Jittering:** this transformation adds to IAT time series N_p different values (one for each packet) extracted from a Gaussian distribution with mean μ and standard deviation σ :

$$\mathbf{IAT}_{jit} \triangleq [\text{iat}_1 \quad \dots \quad \text{iat}_{N_p}] + [\rho_1 \quad \dots \quad \rho_{N_p}],$$

where $\rho_i \sim \mathcal{N}(\mu, \sigma)$, $i = 1, \dots, N_p$.

- **Translation:** consists in adding to all IATs in a biflow the same value extracted from a Gaussian distribution with mean μ and standard deviation σ :

$$\mathbf{IAT}_{tra} = [\text{iat}_1 \quad \dots \quad \text{iat}_{N_p}] + [\rho \quad \dots \quad \rho],$$

where $\rho \sim \mathcal{N}(\mu, \sigma)$.

It should be noted that the transformations illustrated above are not applied to the first packet of each biflow, whose non-scaled value is always constrained to $\text{iat}_1 = 0$ by definition. Furthermore, after each transformation, values exceeding the maximum IAT value (in the training set) are saturated to 1 (i.e., the normalized maximum value).

3.2. Training procedure

Another key aspect for incremental models is the training procedure, regulated by a loss typically composed of several terms and a validation set consisting of new apps biflows. In the following, we introduce the design choices pertaining to (i) the selection of the loss function and (ii) the composition of the validation set.

Remarks on complexity. Since all the investigated modifications for the loss function do not conceptually involve the addition of computationally-expensive operations compared to those proposed in BiC, the resulting complexity is negligibly impacted. Similarly, despite the suggested modification for the validation set results in a set with a different number of samples compared to that used by BiC, this has only a marginal impact on computational complexity. Indeed, the evaluations on the validation set marginally affect the CIL procedure complexity.

3.2.1. General compositional loss

All the CIL methods considered, during their incremental update, face the optimization of a (composite) loss function that has the generic form $\mathcal{L}(\theta)$.

The proposed CIL framework relies on a *composite loss function* (made of different terms) and takes advantage of a memory of old apps biflows (obtained through a *selection strategy*) along with the new apps samples. **The loss function** is composed of *two conflicting* terms:

$$\mathcal{L}(x) = \lambda_{\text{class}} \mathcal{L}_{\text{class}}(x) + (1 - \lambda_{\text{class}}) \mathcal{L}_{\text{dist}}(x), \quad (1)$$

by coupling a *classification* loss (either a *CCE* or a set of *BCE*) with *KD* [32] loss. In detail, while the **classification term** enforces the new (incremental) knowledge into the model (promoting its plasticity), the **distillation term** tends to force the model to maintain the previously learned knowledge and avoid weight drift in the incremental training stage (promoting its stability). The two terms are balanced through the hyperparameter $\lambda_{\text{class}} \in (0, 1)$ as a convex combination. Details for each objective are provided next. First, the classification loss is defined as follows:

$$\begin{aligned} \mathcal{L}_{\text{class}} &= \sum_{x \in D^{\text{mem}}} \text{CCE}(x, y; C) + \sum_{x \in D^{\text{new}}} \text{CCE}(x, y; C, T_{\text{class}}) \\ &= \sum_{x \in D^{\text{mem}}} \sum_{c \in C} -t_c \log p_c(x; C) + \sum_{x \in D^{\text{new}}} \sum_{c \in C} -t_c \log p_c(x; C, T_{\text{class}}). \end{aligned} \quad (2)$$

Note that the *CCE* in the above formula is computed using all the model heads (i.e., old and new) for all biflows. In addition, only for new apps biflows, a temperature factor (T_{class}) is used to smooth the model response to these biflows. Differently, the distillation loss is defined as follows:

$$\mathcal{L}_{\text{dist}} = \sum_{x \in D} \text{CCE}(x, p^*(x; C^{old}, T_{\text{dist}}); C^{old}, T_{\text{dist}}), \quad (3)$$

where $p^*(\cdot)$ indicates the soft outputs obtained by feeding a generic sample x to the previous-episode model. In the latter case, the *CCE* is computed by taking into account only the old model head and employs a temperature factor (T_{dist}). This factor is applied to scale the logits of the current model pertaining to the old apps and the logits of the previous model.

KD enhances generalization by transferring knowledge from a pre-trained model (teacher) to a target network (student), aligning logits, output distributions, intermediate activations, or attention maps [41]. In CIL scenarios, the model trained in the previous episode serves as the teacher, and the student network, starting from the teacher's weights, is the model to be learned in the current episode. Notably, new app

biflows (D^{new}) are incorporated to maintain the “old” model behavior throughout this process.

In our approach, we optimize the loss function using Stochastic Gradient Descent (SGD), a widely used optimization technique for training deep neural networks. Notably, SGD employs the term momentum to accumulate the gradient of the past steps to determine the direction to go. Specifically, SGD with momentum remembers the update $(\Delta\theta)_{(i)}$ at each iteration, and determines the next update as a *linear combination* of the (i) gradient $-\nabla\mathcal{L}(\theta_n)$ — and (ii) the previous update $-(\Delta\theta)_n$ — namely: $\theta_{n+1} = \theta_n - \eta \nabla\mathcal{L}(\theta_n) + \alpha (\Delta\theta)_n$, where η represents the usual learning rate whereas α is used to denote the exponential decay factor. Clearly, $\alpha = 0$ simplifies into the standard SGD with no momentum. In [15,21] a momentum value of $\alpha = 0.9$ is recommended.

In the considered compositional loss, we propose a **Selective Distillation Smoothing** (SDS) procedure to improve the learning procedure. SDS is applied to new apps biflows during the distillation process, with the aim of *softening* the output of the model. SDS strategy is reported in the following formula:

$$p^*(x; C^{old}, T_{dist}) = (1 - \epsilon) \cdot p^*(x; C^{old}, T_{dist}) + \frac{\epsilon}{|C^{old}|} \mathbf{1}_{|C^{old}|}, \quad (4)$$

with ϵ falling in the range $[0, 1]$, where $\epsilon = 0$ implies no smoothing is applied, and $\epsilon = 1$ enforces the uniform distribution.

As an alternative, we also evaluate the effectiveness of **Logits Matching** (LM) loss in the objective function, proposed in [42].

This strategy avoids the information loss occurring in probability space due to the squashing function (e.g., softmax) by computing the Mean Squared Error between the logits pre- and post-incremental update. We examine two options: (i) employing LM loss as $\mathcal{L}_{dist}$, and (ii) combining LM loss with previously-defined $\mathcal{L}_{dist}$.

3.2.2. Balanced validation set

Most of the CIL approaches leverage rehearsal techniques that consist of storing biflows of previous episodes to re-use them in the current training procedure to refresh past knowledge. Typically, the old biflows are poured into the training set of the current episode to be employed in the new training stage.

Generally, in DL, a key role in the training procedure is played by the validation set. It leads the training, drives early stopping mechanisms, and is the yardstick in the evaluation of the best model [43].

In general terms, including old apps biflows in the validation set, the model (during the training) is evaluated to find a point that fits well for old and new apps altogether, instead of taking care only of new ones. Indeed, considering only new apps' biflows could lead to better performance for new apps, but an overall performance degradation.

For this reason, we propose to take advantage of stored biflows for the validation phase as well. To this end, the memory is divided into two portions: 90% training set and 10% validation set. By adopting this approach, we obtain a balanced (per-class) validation set, whose size is $(|D^{mem}|/10 \cdot |C^{old}|) \cdot (|C^{old}| + |C^{new}|)$, encompassing an equal number of biflows for both old and new apps. This contrasts with the validation set employed by BiC that only includes $(|D^{new}|/10)$ biflows of new apps.

3.3. Model rectification

A known concern in the CIL paradigm consists in the *residual bias* of the model toward the most recent apps on which it is trained [33], despite the presence of terms in the incremental loss which are conceived to retain previous knowledge (Eq. (3)). These strategies are known as *Model Rectification*. In this work, we assume that the classification function (viz., the head) is the usual affine transformation defined by a fully-connected (FC) layer, namely $h(\phi) = W^T \phi + b$. Still, for simplicity of exposition, in the following we assume $b = 0$ to better underline the insight into the effect induced by correction weights. A slightly more intricate effect can also be observed in the case of $b \neq 0$.

In detail, depending on what part of the model they operate, three different subgroups can be identified [35]: *Feature*, *Weight*, and *Logit*

Rectification. Our proposal is based on the in-depth exploration of the latter two families, as they both imply in most proposals the same rectification effect on logits, as shown in what follows. Specifically, in this work we evaluate several solutions [33,44,45] proposed in the literature to tackle this abnormal behavior. It is worth underlining that these techniques do not take part in the model training phase, but operate directly in a *post-training* phase by either a heuristic rationale (i.e., in a non-trainable fashion) or by means of a dedicated *correction* set (i.e., in a trainable fashion). For notational convenience, in what follows the logit vector $o(x)$ is cast as

$$o(x) = \begin{bmatrix} o^{old}(x) \\ o^{new}(x) \end{bmatrix}$$

to separate terms associated to old and new classes. Notably, model outputs (before softmax) are obtained through a scalar product between the feature and weight vectors:

$$o_i(x) = \langle \phi(x), w_i \rangle = \|\phi(x)\| \cdot \|w_i\| \cdot \cos(\text{angle}(\phi(x), w_i)), \quad (5)$$

where $\phi(x)$ represents the feature vector, w_i the weight vector of the i th app and $\text{angle}(v_1, v_2) \triangleq \langle v_1, v_2 \rangle / (\|v_1\| \|v_2\|)$ is the angle formed between the two arrays, i.e., their cosine distance.

In our proposal, we introduce a **general correction layer**, referenced with the shorthand notation $\text{CoL}(\alpha^{old}, \alpha^{new}, \beta)$, and defined as follows:

$$\bar{o}(x) = \begin{bmatrix} \alpha^{old} o^{old}(x) \\ \alpha^{new} o^{new}(x) + \beta \cdot \mathbf{1}_{|C^{new}|} \end{bmatrix} = \begin{bmatrix} \alpha^{old} (W^{old})^T \phi(x) \\ \alpha^{new} (W^{new})^T \phi(x) + \beta \cdot \mathbf{1}_{|C^{new}|} \end{bmatrix}, \quad (6)$$

where $\bar{o}(x)$ denotes the output of the correction layer (viz., the adjusted logits vector). Conversely, W^{old} and W^{new} are the matrices collecting the weight vectors associated to old and new classes, i.e., $W = [W^{old} \ W^{new}]$. The first equality in Eq. (6) defines how old and new logits are explicitly scaled, whereas the second equality underlines how this scaling implies an *equivalent norm scaling* on the weight vectors of the FC layer. Indeed, the considered logit rectification can be also interpreted as a weight vector rectification equal to $\|w_i\| \rightarrow \alpha^{old} \|w_i\|$ for each class $i \in C^{old}$. Similarly, a scaling $\|w_i\| \rightarrow \alpha^{new} \|w_i\|$ is applied for each class $i \in C^{new}$. This underlines the *functional equivalence between weight and logit rectification techniques*.

It is worth noticing that the correction layer in Eq. (6) comprises different rectification techniques in literature [33,44] as special instances, as explained hereinafter.

In the present proposal, the three parameters of $\text{CoL}(\alpha^{old}, \alpha^{new}, \beta)$ can be obtained in a *trainable fashion*. More specifically, a few samples from both old and new apps samples are withheld from the overall training set ($D = D^{mem} \cup D^{new}$), thus creating a D^{CoL} and D^{FT} . The former is a class-balanced *correction dataset* with an equal number of biflows for old and new apps whereas the latter is composed of the remaining training biflows ($D \setminus D^{CoL}$). The incremental training then operates as follows. First, the model is fine-tuned using D^{FT} while suppressing the correction layer, i.e., $\text{CoL}(1, 1, 0)$. Then, the whole model parameters are frozen, and the correction parameters $(\alpha^{old}, \alpha^{new}, \beta)$ are tuned re-running the fine-tuning using D^{CoL} and CCE loss. Since the bias correction layer is made of at most three parameters only, it can be fast optimized using a few samples (e.g., a D^{CoL} corresponding to 10% of D size).

In the following, we introduce three model rectification strategies proposed in the CV literature. Accordingly, when applicable, we provide a mapping to our proposed general correction layer $\text{CoL}(\alpha^{old}, \alpha^{new}, \beta)$.

- **Fine-Tuning initialization** (FT_{init}) [45], exploits the FC weights of the previous learning stage for classifying old classes. For instance, after the incremental training, FT_{init} replaces the current FC weights for C^{old} with the corresponding weights from the “previous” model.

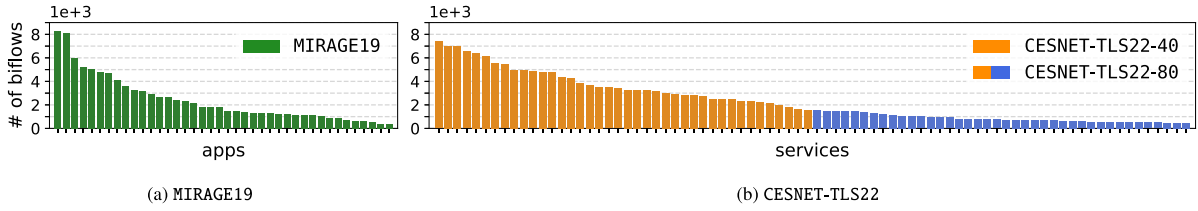


Fig. 2. Datasets composition in terms of number of biflows for each app or service.

Table 2

BiC variants mapping on $\text{CoL}(\alpha^{old}, \alpha^{new}, \beta)$. Gear icon (⚙️) indicates trainable parameters.

	$\text{CoL}(\alpha^{old}, \alpha^{new}, \beta)$
BiC	1 ⚙️ ⚙️
BiC-noCorr	1 1 0
BiC-alt	⚙️ 1 ⚙️
BiC-scale	⚙️ ⚙️ 0
BiC-gen	⚙️ ⚙️ ⚙️

- Weight Aligning (WA) [44], suggests a method to balance the weights of the FC layer for C^{old} and C^{new} to “normalize” new weights and maintain fairness in classification. The underlying assumption is that new weight vectors have higher magnitudes when compared with old ones. This correction is obtained by setting $\text{CoL}(1, \alpha^{new}, 0)$, where the parameter α^{new} (Eq. (7)) is defined in a *non-trainable* fashion by relying on the average ℓ_2 -norm of new and old weight vectors:

$$\alpha^{new} = \frac{\frac{1}{|C^{old}|} \sum_{i \in C^{old}} \|w_i\|}{\frac{1}{|C^{new}|} \sum_{i \in C^{new}} \|w_i\|}. \quad (7)$$

- Bias Correction (BiC) [33], leverages a *trainable* correction layer to mitigate bias toward the new classes. This linear correction layer is obtained by setting $\text{CoL}(1, \alpha^{new}, \beta)$. This layer adjusts the new head logits to re-balance the predictions of old and new apps. As previously mentioned, such a correction layer is trained in a separate training stage with a *correction set* D^{CoL} composed of the same number of samples for all the classes.

Based on the BiC correction, we can specialize other *two* more straightforward strategies by nullifying each time the effect of a single parameter, namely $\alpha^{new} = 1$ and $\beta = 0$. As a result, we obtain the two following special variants of the correction strategy, defined as *BiC-alt*, i.e., $\text{CoL}(\alpha^{old}, 1, \beta)$, and *BiC-scale*, i.e., $\text{CoL}(\alpha^{old}, \alpha^{new}, 0)$. Furthermore, we introduce a more general implementation of the correction layer, namely $\text{CoL}(\alpha^{old}, \alpha^{new}, \beta)$, to which we refer to as *BiC-gen*. It is worth noting that by considering $\text{CoL}(1, 1, 0)$ the effect of the correction is nullified. We will refer to this version as *BiC-noCorr*.

Such variants of the correction proposed in BiC are summarized in Table 2.

Remarks on complexity. Regarding these alternative correction strategies, the increase in complexity can be considered marginal or negligible compared to the specific variant proposed by BiC, since either the same number of parameters is trained, or one additional parameter is introduced (in the case of *BiC-gen*).

4. Experimental setup

In this work, we investigate single-episode increments by considering two different types of scenario according to the number of old and newly added apps. In detail, we define small-increment scenarios (39+1 and 79+1) that emulate the addition of a single app to a model having a

larger set of base classes (39 or 79, respectively) and big-increment scenarios (20+20 and 40+40) that represent the addition of multiple apps, equal to the number of base classes (20 or 40, respectively). Furthermore, we define a multi-increment scenario (20+20+20+20), spanning a total of 4 increments, simulating the addition of apps at different points in time. Results are expressed in terms of average and standard deviation and are obtained considering 40 (or 20 for big-increment and multi-increment scenarios) different random selections of apps (referred to as *runs*). In detail, each run is characterized by a unique permutation of the apps. In other words, each run differs from the other in the order in which each app joins the CIL procedure. This procedure aims at strengthening the significance of measured TC performance.

Unless otherwise stated, we used memory of 1k exemplars for all runs. Our approach is implemented by leveraging the FACIL framework [21] and the following training parameters, namely 200 epochs with patience of 20, the initial learning rate of 0.1, the learning rate decay factor of 10, batch size of 64, and the minimum learning rate of 10^{-7} . Regarding computational resources leveraged for the experimental campaign, all the analyses – including the memory selection strategy and the DL model training – have been executed on a machine with 2 Intel(R) Xeon(R) Gold 6252N CPU @ 2.30 GHz and 378 GB of memory.

4.1. Datasets and input definition

In this work, MIRAGE19 [8] and CESNET-TLS22 [22] datasets have been exploited.

MIRAGE19 contains traffic related to 40 Android apps belonging to 16 different categories.¹ It was collected at the ARCLAB laboratory of the University of Napoli “Federico II” based on multiple measurement campaigns running between May 2017 and May 2019. Overall, the MIRAGE19 dataset contains $\geq 4,600$ PCAP traces – each generated in a capture session of 5–10 min – and $\approx 100k$ labeled biflows distributed across apps, as shown in Fig. 2(a). Although the number of capture sessions for each app is comparable, biflows’ distribution is heavy-tailed with the top-10 (resp. top-20) apps accounting for 54.2% (resp. 78.9%) of all biflows. After discarding zero-payload packets (corresponding to pure signaling), the average length of biflows,² across the dataset corresponds to 180 packets.³

CESNET-TLS22 comprises encrypted traffic data from 191 services categorized under 21 distinct categories.⁴ This dataset was collected during the initial two weeks of October 2021 using high-speed probes located at the perimeter of the CESNET2 network. In its entirety, CESNET-TLS22 comprises 140 million flow records, obtained from various 100 Gb/s network links. The dataset was annotated using TLS information derived from the TLS plugin, which extracts SNI domains from TLS handshakes. To capitalize on the CESNET-TLS22 dataset, we

¹ <https://traffic.comics.unina.it/mirage/mirage-2019.html>.

² In network-traffic-classification field, the most commonly used traffic object is the biflow – i.e., *bidirectional* flow – constituted by packets identified by a quintuple (*source IP*, *source port*, *destination IP*, *destination port*, and *transport protocol*), where *source* and *destination* can be swapped.

³ It is worth noticing that while 63% of biflows have size ≥ 10 packets, only 22% has ≥ 50 packets.

⁴ <https://zenodo.org/record/7965515>.

have devised two downsampled versions of it: (i) CESNET-TLS22-40, which consists of the top 40 represented services in CESNET-TLS22, used to evaluate the proposal on the same scenarios as the MIRAGE19 dataset, and (ii) CESNET-TLS22-80, which includes the top 80 represented services in CESNET-TLS22, used to evaluate a more challenging scenario by doubling the number of classes available in MIRAGE19. Additionally, we resampled each class composing CESNET-TLS22 by randomly selecting 0.15% of biflows, resulting in a number of samples comparable to MIRAGE19 (see Fig. 2). This decision was motivated by the objective of guaranteeing a balanced and fair comparison between the two datasets. Indeed, by equalizing the class distribution of the two datasets, we can accurately assess the performance of CIL approaches without the bias introduced by different dataset sizes, allowing a more meaningful and precise evaluation.

In this work, *four* per-packet properties are considered, namely transport-layer payload size (PS), inter-arrival time between consecutive packets (IAT), TCP window size (WIN), and direction (DIR). It should be noted that the CESNET-TLS22 dataset does not provide the WIN field. Accordingly, for CESNET-TLS22-40 and CESNET-TLS22-80 we leveraged *three* per-packet properties, namely PS, IAT, and DIR.

In previous works [6,46], it has been found that such per-packet properties extracted from the sequence of the first 10 packets (without considering zero-payload ones) of each biflow are sufficient to attain good classification performance without sacrificing the “earliness” of the traffic classifier. Thus, for each biflow, time series of length 10 are extracted for the considered per-packet properties. These are then used to feed the DL classifiers considered. Detailing the encoding, DIR is set to +1 for upstream or -1 for downstream packets, while WIN is set to zero for UDP packets. In case the time series are shorter than 10 packets, zero-padding is applied.

4.2. Model architecture

In this work, we leverage a 2D-CNN with two convolutional layers (Conv2D) followed by pooling and normalization operations and a Dense layer that provides the model output. Actually, CNN-based models are commonly used in SOTA approaches for TC [6,7,47]. This architecture has $\approx 540k$ trainable parameters overall.

Specifically, the network is fed with $\#packets = 10$ and $\#fields = 4$ per biflow ($\#fields = 3$ for CESNET-TLS22).

At each incremental iteration, the structure of the backbone remains unchanged while the head – i.e., the last layer of the network – is expanded with as many neurons as new classes are added in that episode. The incremental training starts from the weights computed in the previous training stage (θ^*).

4.3. CIL baselines

To evaluate the performance benefits of MEMENTO, we consider as baselines a Lower Bound (LB) and BiC. Indeed, the latter approach was shown to outperform a large pool of SOTA fine-tuning approaches on MIRAGE19 [20] for both the small-increment (39+1 apps) and big-increment (20+20 apps) scenarios. This finding has also been confirmed in different application domains, e.g., cybersecurity [48], where BiC outperforms all the SOTA CIL approaches considered in the detection of zero-day (viz., novel) attacks.

In detail, we define as LB the model trained from scratch using the training data subset corresponding to $\mathcal{D} = \mathcal{D}^{mem} \cup \mathcal{D}^{new}$. Given the existing model trained on $\mathcal{C}^{old}$, we first select the memory biflows via herding selection. Then, an extended model is trained (from scratch) by leveraging new apps data and memory biflows. This approach represents the most basic way to achieve a model that deals with both $\mathcal{C}^{old}$ and $\mathcal{C}^{new}$ under memory constraints, namely with a small amount of biflows for $\mathcal{C}^{old}$ classes. In addition, LB avoids employing advanced mitigation strategies to address issues such as forgetting and intransigence phenomena.

Differently, BiC is a sophisticated fine-tuning approach that adopts a two-term loss ($\mathcal{L}_{class} = |\mathcal{C}^{old}|/|C|$ in Eq. (1)) and introduces a new linear layer after the model head, namely the *bias correction layer*, described in Section 3.3. In general, BiC requires four steps: (i) collection of distillation probabilities, (ii) model update, (iii) fine-tuning of the bias correction layer and (iv) herding selection. Notably, BiC leverages the same $\mathcal{L}_{class}$ (Eq. (2)) without temperature scaling on new classes, i.e., $T_{class} = 1$. Instead, regarding $\mathcal{L}_{dist}$ (Eq. (3)) BiC presents $T_{dist} = 2$.

4.4. Evaluation metrics

In this section, the metrics used to evaluate CIL approaches are detailed, also providing the relevant rationale behind them. In detail, all the considered metrics are broken down by the subset of classes $\mathcal{C}^{new}$, $\mathcal{C}^{old}$, and $\mathcal{C}^{all}$. Hence, for a generic metric X , three sub-metrics X^{old} , X^{new} , and X^{all} are obtained accordingly. This is to analyze separately the trade-off between plasticity ($\mathcal{C}^{new}$) and stability ($\mathcal{C}^{old}$) of CIL methods and to evaluate their combined effect on the updated (viz., incremented) traffic classifier ($\mathcal{C}^{all}$). In what follows, we introduce the two complementary metrics used in the evaluation of this work.

First, the *F1 Score* is considered as the relevant performance measure, following previous studies [14]. This metric considers both (per-class) recall and precision via their harmonic mean and then takes the arithmetic average (viz., macro F1-score) over all the apps, namely

$$F1\ Score(\theta, C) = \frac{1}{|C|} \sum_{i \in C} \frac{2 \cdot Precision_i \cdot Recall_i}{Precision_i + Recall_i}, \quad (8)$$

where C is used to denote the generic app set and θ a generic model (learned either from-scratch or in incremental fashion).

Due to the interplay of precision and recall of a given app, F1 Score hides the underlying behavior of the model and hinders the grasp of forgetting phenomena causes (whether catastrophic forgetting or intransigence are occurring). For this reason, in this work, the evaluation is extended with a suitable metric to better understand and explain the behavior of approaches in the CIL scenario. Specifically, the second metric introduced to evaluate and troubleshoot the experimental results is the *Balanced Accuracy*, which is defined as follows:

$$Balanced\ Accuracy(\theta, C) = \frac{1}{|C|} \sum_{i \in C} Recall_i. \quad (9)$$

We recall that the Balanced Accuracy choice is further motivated by robustness considerations in the case of class imbalance and/or in CIL scenarios with a large disproportion between number of base and new classes (thus allowing an informative interpretation of the overall metric).

Drop from Scratch evaluation: To evaluate the effectiveness of the incremental learning process, each metric of CIL-based traffic classifier needs to be compared against the same value calculated on a trained-from-scratch model, rather than considering its absolute value. Therefore, we define the *drop from Scratch* to assess the various aspects of performance loss from the upper bound. For this metric, *lower values are better*. Accordingly, given a performance metric X (e.g., F1 Score, Balanced Accuracy), the definition of X Drop is as follows

$$X\ Drop(C) = X(\theta^{Scratch}, C) - X(\theta^{CIL}, C), \quad (10)$$

and according to the considered set of apps, three views are considered: DropAll = Drop(C_m), DropOld = Drop(C_m^{old}), and DropNew = Drop(C_m^{new}).

Maximum Improvement Over the Best Baseline (MIOBB): To assess the improvement over the top-performing baseline, each metric X attained by the proposed solution (MEMENTO) is compared with the corresponding value exhibited by the Best Baseline (BB). The latter baseline is selected based on the lowest F1 Score Drop All. This metric is reported according to the considered set of apps (e.g., All, Old, New) and it is defined by the following formula:

$$X\ MIOBB(C) = X(\theta^{Memento}, C) - X(\theta^{BB}, C). \quad (11)$$

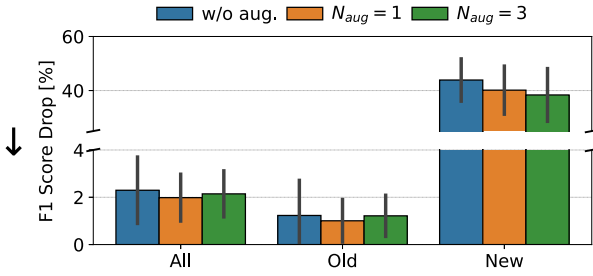


Fig. 3. Comparison of F1 Score Drops for BiC without and with augmentation (aug.). The arrow (l) indicates that the lower the drops are, the better performance is.

Table 3

Comparison of different selection Strategies. Sorted by decreasing F1 Score Drop All. Lower is better (reported in **bold**). The darker the row color, the more complex the strategy. In the *Completion Time* column, the time required by each strategy is reported in seconds and expressed as a percentage of the total time required by the incremental procedure in brackets.

Selection Strategy	F1 Score Drop [%]			Completion time [s]
	All	Old	New	
Herding	2.30 ± 1.40	1.23 ± 1.48	43.89 ± 7.82	≈ 40 (≈ 15%)
Rainbow	2.62 ± 4.15	1.58 ± 4.19	43.28 ± 8.83	≈ 320 (≈ 58%)
Random	2.72 ± 4.70	1.68 ± 4.75	43.33 ± 8.99	< 10 (< 4%)
Distance	2.87 ± 2.07	1.71 ± 2.12	48.42 ± 13.32	≈ 10 (≈ 4%)
Entropy	2.91 ± 1.96	1.76 ± 2.00	47.76 ± 12.87	≈ 10 (≈ 4%)

5. Experimental evaluation of building blocks

In the following, the results of the experimental evaluation are reported and discussed, focusing on *each specific building block*. More specifically, Section 5.1 investigates the trade-off between performance and time of different sampling strategies and puts forward traffic biflows augmentation primitives. Then, Section 5.2 assesses the considered training procedure, including the distillation-classification dilemma in the composite loss, the identification of a suitable loss for distillation and the need for defining a validation procedure to lead effectively the training process. Last, Section 5.3 investigates the considered general rectification strategy and compares it with existing alternatives.

All the analyses presented below aim to formulate a new CIL approach, named MEMENTO. This approach will incorporate the best strategies identified through the experiments presented in this section for each of the building blocks. We underline that we consider BiC as the starting point (best fine-tuning CIL approach [20]) for our experiments. For brevity, our analyses of the three building blocks composing MEMENTO (Sections 5.1–5.3) are carried out in 39+1 scenario for MIRAGE19 dataset.

5.1. Memory management

Table 3 shows the performance in terms of F1 Score Drop – for all, old, and new apps – obtained employing different selection strategies in conjunction with BiC. From inspection of the results, we observe that herding, random, and rainbow memory exhibit better performance with respect to distance- and entropy-based methods. In detail, despite the random selection and rainbow memory reach similar performance w.r.t. herding on C^{new} , herding achieves a non-negligible improvement on C^{old} .

Furthermore, we conduct a time analysis⁵ to assess the optimal selection strategy, presenting results in both seconds and as a percentage of the total time for the incremental procedure, covering memory

selection and training. In detail, the strategy with the shortest execution time is evidently the random approach, as it does not involve any additional operations for sample selection and accounts for ≤4% of the incremental process. Conversely, distance- and entropy-based strategies each require ≈10 s (≈4% of the incremental process). The herding strategy contributes ≈40 s, representing 15% of the entire incremental procedure. Notably, the rainbow memory strategy proves the most time-consuming, requiring ≈320 s, exceeding 50% of the entire incremental process. It is worth underlining that a drawback linked to the random selection strategy originates from its nature: the selection of exemplars is not deterministic (as opposed to the others). This increases the chance that the outcome might result in a particularly unfavorable memory. Additionally, the memory selection process can be parallelized, allowing for the concurrent selection of memory samples for each class and, consequently, reducing the overall selection time.

As a result, in the following analysis, we leverage herding as the (deterministic) biflows selection strategy since it turns out to be the best trade-off between time and performance.

To further capitalize on the memory, we applied the two traffic-data augmentation strategies defined in Section 3.1. In detail, we set for both the primitives $\mu = 0$ and $\sigma = 0.03$. Following these procedures, we generate (for each biflow in the memory) N_{aug} biflows with jittering and N_{aug} biflows with translation, thus obtaining an *augmented set* of $(2N_{aug} + 1)$ biflows (viz., $2N_{aug}$ synthetic biflows are added to each original biflow stored in memory $\mathcal{D}^{mem}$). To assess the benefits of biflows augmentation independently, we measure the impact of this procedure on the BiC baseline. We experimentally evaluate performance when N_{aug} is equal to 1 (thus resulting in a 3× augmentation) and to 3 (viz., 7× augmentation). We experimented with these N_{aug} values to minimize the effect on training duration. Accordingly, results shown in Fig. 3 highlight that the enforcement of data augmentation for the memory has no major impact on old apps, but is *highly beneficial for new apps*, improving performance figures for F1 Score Drop. Quantitatively, the F1 Score Drop for the new class with 3× (resp. 7×) augmentation is reduced by ≈4% (resp. ≈6%) and reaches a value of 40% (resp. 38.40%).

To complement the aforementioned analysis, the time needed for training BiC in (a) its base configuration (w/o aug.) and (b) with 3× and 7× augmentation has been analyzed. The results are scaled by the training time of a scratch model and are averaged across 40 runs. With the use of 3× (resp. 7×) biflows augmentation for the old apps, the average training time increases by ≈1.5× (resp. ≈2.5×) compared to the basic configuration of BiC using only memory biflows for old apps.

In light of these observations, we select the 3× augmentation as the best trade-off between performance and training time.

5.2. Training procedure

In the following, we report the results for enhancements regarding the training procedure, assessing the impact of *momentum*, the various components of the *compositional loss*, and the effect of the *balanced validation set*.

First, we compare the performance obtained using a momentum $\alpha = 0.9$ (standard value in TC literature [15]) with a simpler counterpart that omits this term (i.e., momentum set to $\alpha = 0$). As a result, we found that the composite loss in CIL should be optimized with a gradient update that does not take into account any momentum accumulation. Indeed, when a momentum of 0.9 is applied, F1 Score DropAll increases to 7.78%, while F1 Score DropOld and F1 Score DropNew reach 6.72% and 49.5%, respectively.

⁵ We underline that the execution time for experiments is measured on a single-core Intel(R) Xeon(R) CPU Gold 6252N @ 2.30 GHz with 378 GB of RAM, with only one experiment running at a time and with no other user

program running. All the execution times have been computed by leveraging the Python `time.process_time()` function to take into account only the actual run-time on the CPUs (i.e., sequentially).

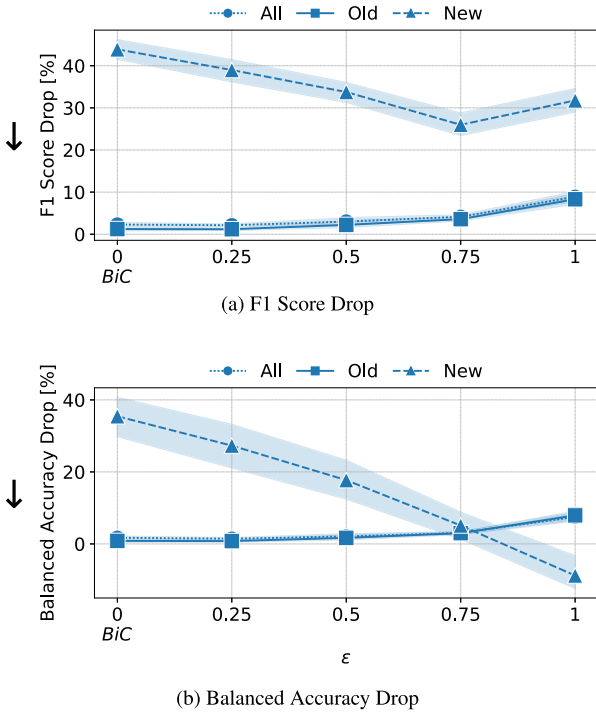


Fig. 4. Analysis of SDS factor in terms of F1 Score Drop (a) and Balanced Accuracy Drop (b). The arrow ($\downarrow$) indicates that the lower the drops are, the better performance is.

Then, we analyze the impact of λ_{class} (cf. Eq. (1)), letting it vary in $\{0, 0.25, 0.5, 0.75, 1\}$ to appreciate the influence of information retention and transfer. We highlight that when $\lambda_{class} = 1$ the classification loss becomes the only contribution to the total loss, whereas when $\lambda_{class} = 0$ the loss function encompasses only the distillation term. Furthermore, we recall that BiC defines a specific value for the λ_{class} parameter: $\lambda_{class} = |C^{new}|/|C|$. The analysis highlights that nullifying the effect of $\mathcal{L}_{class}$ (i.e., $\lambda_{class} = 0$) results in a zero F1 Score for C^{new} . In the configuration defined by BiC, significantly higher results are achieved. Then, increasing λ_{class} (thus reducing $\mathcal{L}_{dist}$), both the F1 Score and Balanced Accuracy Drop for C^{old} increase (up to $\approx 20\%$ for F1 Score and 15% for Balanced Accuracy when $\lambda_{class} = 1$), whereas the drop for C^{new} remains quite stable for all the values of λ_{class} . In conclusion, the BiC default value for λ_{class} (i.e., $|C^{new}|/|C| = 0.025$ for our evaluation scenario) seems to represent a reasonable candidate and is taken as the default value for our analysis.

Furthermore, we investigate the performance when $\lambda_{class} = |C^{new}|/|C|$ by varying the temperature T_{dist} . In our experiments, we start with $T_{dist} = 1$ (no temperature scaling) and arrive at $T_{dist} = 2$, employed by BiC, with a step size of 0.25. The initial configuration ($T_{dist} = 1$) shows the worst performance with F1 Score DropAll = 4.05%, F1 Score DropOld = 2.87% and F1 Score DropNew = 50.13%. Then, results gradually improve and the best performance is reached when T_{dist} is set to 2.

As shown in Eq. (2), also the categorical cross-entropy loss used to learn new apps is characterized by the presence of a temperature parameter, namely T_{class} . Such knob is employed to adjust the learning effect on new apps and is here evaluated by varying the aforementioned parameter in $\{1, 2, 3, 4, 5\}$.

In detail, while the DropOld of both F1 Score and Balanced Accuracy remains stable, the DropNew of both metrics experiences a significant improvement of 10.06% F1 Score (resp. 11.92% Balanced Accuracy) moving from $T_{class} = 1$ (no temperature effect) to $T_{class} = 3$. Notably, for $T_{class} > 3$ performance improvement tends to saturate. Hence,

we selected $T_{class} = 3$ as the relevant trade-off value for the following analyses.

As illustrated in Section 3.2, the proposed loss introduces the SDS technique to soften the old model response for samples of the new app in the distillation phase. To assess the effectiveness of this technique in our CIL scenario, we perform a sensitivity analysis for the smoothing parameter as $\epsilon \in \{0, 0.25, 0.5, 0.75, 1\}$. While 0 corresponds to the standard BiC approach, selecting 1 means forcing a uniform distribution ($1/|C^{old}|$) for the old apps on the samples $x \in \mathcal{D}^{new}$. As depicted in Fig. 4, the application of SDS improves new app performance while slightly decreasing performance on old apps.

Concerning F1 Score (Fig. 4(a)), suitable ϵ values can be identified in $\epsilon = 0.25$, where DropNew reaches $\approx 39\%$ ($\approx -5\%$ w.r.t. BiC), and $\epsilon = 0.5$ with a DropNew of $\approx 34\%$ (about -10% w.r.t. BiC). Both such configurations present very small values of DropOld ($< 2.5\%$). Conversely, the ever-improving trend shown by the Balanced Accuracy DropNew (Fig. 4(b)) is indicative of decreasing precision on new apps. Hence, looking at both the metrics, the best trade-off between DropNew and DropOld is $\epsilon = 0.5$.

As the last analysis for the compositional loss, we compare BiC distillation loss (namely, KD) and LM loss. First, we contrast the two losses separately, i.e., using either of them for the distillation term in Eq. (1). Also, we combine them in a two-term distillation loss where each term is weighed half. As a matter of fact, employing only LM (instead of KD) leads to a substantial degradation across all three sets of apps ($\approx +15\%$ F1 Score DropAll and F1 Score DropOld, $\approx +18\%$ F1 Score DropNew). On the other hand, when LM and KD are combined, performance shows improvement w.r.t. employing LM only. Nevertheless, the sole use of KD still provides the best performance ($\approx +2\%$ F1 Score DropAll, $\approx +1\%$ F1 Score DropOld, and $\approx +7\%$ F1 Score DropNew). To sum up, KD offers the best performance for C^{old} , C^{new} , and C^{all} with respect to LM.

Finally, we assess the performance shift when considering a balanced validation set composed of a balanced number of biflows per app. Employing the proposed balanced configuration for the validation set is beneficial for C^{new} as F1 Score DropNew is reduced by 3% with respect to BiC, whereas F1 Score DropOld remains quite stable ($\approx 0.1\%$ reduction w.r.t. BiC). Balanced Accuracy exhibits the same reductions of F1 Score.

As a result, it is worth noticing that introducing a balanced validation set has the benefit of a less sharp transition between forgetting and intransigence. Equally important, the use of part of the memory for the validation does not seem to harm its use in the main training according to the defined loss.

5.3. Model rectification

In this section, we assess various techniques to rectify logits and repair the residual bias of the model.

Fig. 5 reports a comparison of the different rectification strategies investigated in this work. All the strategies are compared with the performance obtained with the BiC approach before performing the correction—BiC-noCorr in the following. This baseline turns out to have very unbalanced predictions toward the new app. This aspect can be seen from Fig. 5(b) where an even negative drop in balanced accuracy is pointed out.

The FT_{init} correction brings minor improvements for the old apps as the norms of scratch weights, that replace the current model weights, typically assume similar (slightly higher) values. Since no correction is applied for the new app, it presents the same phenomenon that involves BiC-noCorr.

As described in Section 3.3, WA is based on the assumption that the mean of the ℓ_2 -norm of new apps weights is higher than the norm of old apps weights. In the investigated scenario, this assumption is not verified for any of the added applications, indeed the ℓ_2 norm of new app weights is higher than the old ones and, therefore, $\alpha_2 > 1$. As a result, the corrective factor raises the weights norm for the new app

Table 4

Comparison of baselines and our proposal MEMENTO in a small-increment scenario (39+1 apps). The last row shows the MIOBB, namely the improvement of MEMENTO_{small} over BiC. On MIRAGE19 (resp. CESNET-TLS22-40), Scratch achieves $\approx 76\%$ (resp. $\approx 88\%$) F1 Score and $\approx 75\%$ (resp. $\approx 88\%$) Balanced Accuracy.

Dataset	Approach	F1 Score Drop [%]			Balanced Accuracy Drop [%]		
		All	Old	New	All	Old	New
MIRAGE19	LB	9.47 $\pm$ 2.24	8.37 $\pm$ 2.32	52.24 $\pm$ 7.67	8.14 $\pm$ 4.2	8.85 $\pm$ 4.33	-19.76 $\pm$ 10.59
	BiC	2.30 $\pm$ 1.40	1.23 $\pm$ 1.48	43.89 $\pm$ 7.82	1.71 $\pm$ 1.26	0.84 $\pm$ 1.36	35.45 $\pm$ 18.09
	MEMENTO _{small}	2.24 $\pm$ 1.02	1.59 $\pm$ 0.91	27.59 $\pm$ 11.26	1.26 $\pm$ 0.91	0.59 $\pm$ 0.86	27.38 $\pm$ 13.41
	MIOBB ^a	+0.06	-0.36	+16.30	+0.45	+0.25	+8.07
CESNET-TLS22-40	LB	9.14 $\pm$ 3.09	8.08 $\pm$ 3.08	50.62 $\pm$ 8.97	9.98 $\pm$ 4.04	10.52 $\pm$ 4.15	-10.95 $\pm$ 9.76
	BiC	1.81 $\pm$ 0.76	0.97 $\pm$ 0.52	34.37 $\pm$ 13.06	1.82 $\pm$ 0.72	1.15 $\pm$ 0.57	27.71 $\pm$ 19.9
	MEMENTO _{small}	1.49 $\pm$ 0.72	0.95 $\pm$ 0.47	22.55 $\pm$ 9.97	1.45 $\pm$ 0.67	1.0 $\pm$ 0.48	18.93 $\pm$ 17.09
	MIOBB ^a	+0.32	+0.02	+11.82	+0.37	+0.15	+8.78

^a Higher is better.

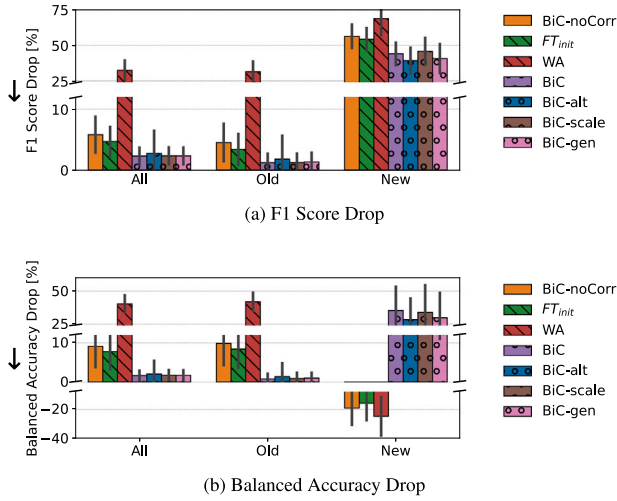


Fig. 5. Comparison of rectification strategies. The standard deviation may be truncated by the visualization, but at least one of the two symmetrical lines is shown. BiC-noCorr (without hatch) does not perform any correction. FT_{init} and WA (stripped hatch) present non-trainable correction, whereas the others (dotted hatch) leverage a correction with a trainable layer. The arrow ($\downarrow$) indicates that the lower the drops are, the better performance is.

and, consequently, the imbalance toward the new app becomes higher than before the correction. For this reason, WA correction leads to the worst F1 Score results for all the three apps subsets— C^{old} , C^{new} , and C^{all} .

The other four strategies depicted in the figure are based on the generalized (trainable) correction layer that includes BiC as a special case. The experimental evaluation shows similar results if evaluated on the whole apps— C^{all} . Going into detail, BiC-gen shows the smaller F1 Score Drop on C^{new} ($\approx 41\%$), with BiC second best ($\approx 44\%$), while these roles are swapped when considering C^{old} ($\approx 1.3\%$ for BiC and $\approx 1.4\%$ for BiC-gen).

In light of these considerations, **we decide to include in MEMENTO the rectification strategy proposed by BiC since it offers the best overall performance** (slightly better than BiC-gen).

We also evaluated the performance obtained by substituting the more common categorical cross entropy with a Multi-class Focal loss in the BiC correction level training, without substantial differences in performance.

6. Definition and Experimental Evaluation of MEMENTO

In the next subsection (Section 6.1), we define MEMENTO for both small-increment and big-increment scenarios as a composition of the

previously-analyzed building blocks. Then, we evaluate MEMENTO with MIRAGE19 and CESNET-TLS22-40 in 39+1 (Section 6.2) and 20+20 (Section 6.3). Then, we assess our proposal with a larger set of apps with CESNET-TLS22-80 in 40+40 and 79+1 scenarios (Section 6.4). Finally, we perform a time analysis to assess the computational complexity of MEMENTO across the considered scenarios (Section 6.5).

6.1. MEMENTO Definition

In the following, we define our novel approach MEMENTO bringing together the enhancements of each building block, briefly reported in the take-home messages of each previous paragraph. The resulting notation for MEMENTO is:

$$\text{MEMENTO}(N_{aug}, \alpha, \lambda_{class}, T_{dist}, T_{class}, \epsilon, bal-val, CoL(\cdot)),$$

where N_{aug} is the number of biflows generated (starting from a memory biflow) through each of the augmentation primitives, α is the momentum enforced, λ_{class} is the balancing factor between $\mathcal{L}_{class}$ and $\mathcal{L}_{dist}$, T_{dist} is the temperature scaling used in KD, T_{class} is the temperature scaling for the distillation on new classes, ϵ is the SDS factor, $bal-val$ indicates if the balanced validation has been used, $CoL(\cdot)$ is the correction layer enforced. To sum up, in 39+1 scenario MEMENTO includes biflows augmentation techniques with $N_{aug} = 1$, no momentum ($\alpha = 0$), $\lambda_{class} = 0.025$, $T_{dist} = 2$, $T_{class} = 3$, SDS with $\epsilon = 0.50$, $T_{class} = 3$, and balanced validation and the same correction as BiC. Furthermore, since the validation set is obtained as 10% of memory biflows, after the augmentation process, we obtain a validation set that is 3 $\times$ the original one. We ensure that each biflows with its derived synthetic biflows falls into the same set (viz., training set or validation set). This configuration will be referred as:

$$\text{MEMENTO}_{small} = \text{MEMENTO}(1, 0, 0.025, 2, 3, 0.5, \text{True}, \text{BiC}).$$

For the big-increment scenario, the imbalance between the number of old and new apps is lower. For this reason, some countermeasures taken to reduce forgetting phenomena are not required in this scenario. On the other hand, other strategies introduced to alleviate catastrophic forgetting and intransigence turn out to be effective also in this scenario. Going into detail, results obtained in 39+1 scenario for momentum, T_{dist} , SDS, and KD (instead of LM) are confirmed also in 20+20 scenario. Concerning λ_{class} and T_{class} , the best performance is obtained with 0.25 (instead of the resulting value 0.50 for BiC) and 1, respectively. The need for a balanced validation set composed of all apps seems to be not apparent. Biflows augmentation seems to be not necessary due to the lower imbalance in biflows number for C^{old} and C^{new} . Finally, the most suitable correction layer to the big-increment scenario proves to be BiC-alt. This configuration will be referred as:

$$\text{MEMENTO}_{big} = \text{MEMENTO}(0, 0, 0.25, 2, 1, 0.5, \text{False}, \text{BiC-alt}).$$

In the following, we compare our proposal with the baselines defined in Section 4.3 by evaluating them in several scenarios.

Table 5

Comparison of baselines and our proposal MEMENTO in a big-increment scenario (20+20 apps). The last row shows the MIOBB, namely the improvement of MEMENTO_{big} over BiC. On MIRAGE19 (resp. CESNET-TLS22-40), Scratch achieves $\approx 76\%$ (resp. $\approx 88\%$) F1 Score and $\approx 75\%$ (resp. $\approx 88\%$) Balanced Accuracy.

Dataset	Approach	F1 Score Drop [%]			Balanced Accuracy Drop [%]		
		All	Old	New	All	Old	New
MIRAGE19	LB	31.56 $\pm$ 1.53	37.69 $\pm$ 2.77	25.42 $\pm$ 2.71	28.72 $\pm$ 1.4	49.26 $\pm$ 2.66	8.18 $\pm$ 1.87
	BiC	11.89 $\pm$ 4.04	11.03 $\pm$ 3.82	12.75 $\pm$ 5.31	10.19 $\pm$ 3.23	9.85 $\pm$ 8.8	10.54 $\pm$ 12.39
	MEMENTO _{big}	8.59 $\pm$ 1.54	7.81 $\pm$ 2.64	9.38 $\pm$ 2.18	7.84 $\pm$ 1.24	10.37 $\pm$ 2.92	5.3 $\pm$ 3.19
	MIOBB ^a	+3.3	+3.22	+3.37	+2.35	-0.52	+5.24
CESNET-TLS22-40	LB	16.68 $\pm$ 1.47	21.55 $\pm$ 2.76	11.82 $\pm$ 2.39	14.66 $\pm$ 1.28	32.45 $\pm$ 3.19	-3.13 $\pm$ 1.52
	BiC	9.16 $\pm$ 5.53	7.58 $\pm$ 3.02	10.74 $\pm$ 9.13	8.57 $\pm$ 4.82	7.89 $\pm$ 5.88	9.25 $\pm$ 13.6
	MEMENTO _{big}	6.62 $\pm$ 0.66	6.19 $\pm$ 1.84	7.05 $\pm$ 2.03	6.28 $\pm$ 0.71	8.74 $\pm$ 2.47	3.81 $\pm$ 2.52
	MIOBB ^a	+2.54	+1.39	+3.69	+2.29	-0.85	+5.44

^a Higher is better.

Table 6

Comparison of baselines and our proposal MEMENTO for CESNET-TLS22-80 in 79+1 and 40+40 scenarios. The last row shows the MIOBB, namely the improvement of MEMENTO_{small} and MEMENTO_{big} over BiC. On CESNET-TLS22-80, Scratch achieves $\approx 84\%$ F1 Score and $\approx 83\%$ Balanced Accuracy.

Scenario	Approach	F1 Score Drop [%]			Balanced Accuracy Drop [%]		
		All	Old	New	All	Old	New
79+1	LB	9.88 $\pm$ 6.91	9.42 $\pm$ 6.91	48.38 $\pm$ 12.10	7.96 $\pm$ 7.89	8.25 $\pm$ 8.00	-11.51 $\pm$ 9.94
	BiC	1.31 $\pm$ 0.60	0.89 $\pm$ 0.42	37.41 $\pm$ 17.63	1.07 $\pm$ 0.65	0.73 $\pm$ 0.63	31.49 $\pm$ 25.47
	MEMENTO _{small}	1.34 $\pm$ 0.52	1.15 $\pm$ 0.42	19.03 $\pm$ 10.67	0.86 $\pm$ 0.51	0.67 $\pm$ 0.43	19.49 $\pm$ 15.84
	MIOBB ^a	-0.03	-0.26	+18.38	+0.21	+0.06	+12.00
40+40	LB	13.72 $\pm$ 0.95	16.48 $\pm$ 1.96	10.96 $\pm$ 2.25	10.72 $\pm$ 0.83	25.10 $\pm$ 2.16	-3.65 $\pm$ 1.44
	BiC	13.56 $\pm$ 8.89	11.91 $\pm$ 6.72	15.21 $\pm$ 12.61	11.07 $\pm$ 7.04	10.54 $\pm$ 12.32	11.61 $\pm$ 19.36
	MEMENTO _{big}	5.87 $\pm$ 0.60	5.51 $\pm$ 1.51	6.23 $\pm$ 1.87	5.01 $\pm$ 0.65	6.04 $\pm$ 2.15	3.98 $\pm$ 2.65
	MIOBB ^a	+7.69	+6.40	+8.98	+6.06	+4.50	+7.63

^a Higher is better.

Table 7

Percentage time gain (w.r.t. Scratch) for BiC and MEMENTO across all scenarios and datasets. Notably, Scratch requires ≈ 3400 s on MIRAGE19, ≈ 4000 s on CESNET-TLS22-40, and ≈ 4900 s on CESNET-TLS22-80.

Scenario	Approach	Time gain [%]	
		MIRAGE19	CESNET-TLS22
39+1	BiC	93.05	93.75
	MEMENTO _{small}	89.38	90.86
20+20	BiC	16.62	11.82
	MEMENTO _{big}	16.62	11.82
79+1	BiC	N.D.	93.82
	MEMENTO _{small}	N.D.	89.09
40+40	BiC	N.D.	11.72
	MEMENTO _{big}	N.D.	11.72

N.B.: CESNET-TLS22 corresponds to CESNET-TLS22-40 in 39+1 and 20+20 scenarios, and to CESNET-TLS22-80 in 79+1 and 40+40 scenarios; N.D. indicates not defined configurations.

6.2. Small-increment scenario

The results and the comparison with the SOTA approaches are reported in Table 4. Our novel proposal MEMENTO_{small} obtains an improvement over the best baseline (BiC) for both the investigated datasets.⁶ In detail, for MIRAGE19 (resp. CESNET-TLS22-40) it exhibits a F1 Score DropNew reduction (MIOBB) of 16.3% (resp. 11.8%), while remaining stable on old and all apps. Similarly, gains are obtained also for

⁶ For the sake of completeness, we examined both BiC and MEMENTO_{small} on MIRAGE19 with the same fields available for CESNET-TLS22—viz., PL, IAT, and DIR. Even with this reduced set of fields, MEMENTO_{small} demonstrates superior performance compared to BiC, achieving a $\approx 16\%$ improvement in F1 Score for C^{new} while maintaining stability for C^{old} and C^{all} .

Balanced Accuracy, with a Drop reduction (MIOBB) of $\approx 8\text{--}9\%$ for new apps for both the datasets. Considering LB, for the MIRAGE19 dataset (resp. CESNET-TLS22-40), the performance gain in terms of F1 Score is $\approx 25\%$ (resp. $\approx 28\%$) on C^{new} and $\approx 7\%$ on C^{old} and C^{all} for both the datasets.

These results clearly show that our proposal MEMENTO_{small} obtains a relevant reduction of intransigence w.r.t. BiC, leaving unaltered the capability of recognizing old classes, and better balancing the old–new knowledge.

6.3. Big-increment scenario

In this section, we report the results related to the big-increment scenario, i.e., considering 20+20 apps. The results are summarized in Table 5, where our proposal MEMENTO_{big} is compared against SOTA approaches, i.e., BiC, and the LB, in terms of F1 Score and Balanced Accuracy Drops by varying the set of considered apps, namely for All, Old, and New apps. The results are shown for both the MIRAGE19 and CESNET-TLS22-40 datasets.⁷ Considering LB, for MIRAGE19 (resp. CESNET-TLS22-40) the performance gain in terms of F1 Score is $\approx 16\%$ (resp. $\approx 5\%$) on C^{new} , $\approx 30\%$ (resp. $\approx 15\%$) on C^{old} , and $\approx 23\%$ (resp. $\approx 10\%$) on C^{all} . For MIRAGE19, MEMENTO_{big} outperforms the best baseline (BiC) by $\approx 3\%$ for all the three subsets of classes, while for the CESNET-TLS22-40 dataset F1 Score improvements are as follows: 3.7% on C^{new} , 1.4% on C^{old} , and 2.5% on C^{all} . As for Balanced Accuracy, for both the datasets, MEMENTO_{big} exhibits a $\approx 5\%$ improvement for new classes, while remaining stable for the old ones, thus obtaining better performance of $\approx 2\%$.

⁷ We assessed MEMENTO_{big} performance on MIRAGE19 by excluding the WIN field. Once more, MEMENTO_{big} outperforms BiC, showcasing F1 Score improvements of 1.4% for C^{all} , 1% for C^{old} , and $\approx 2\%$ for C^{new} .

Table 8

Comparison of baselines and our proposal MEMENTO for CESNET-TLS22-80 in 20+20+20+20. In detail, we evaluate performance in terms of F1 Score Drop for the three intermediate scenarios [20]+20 (named ep. 1), [20]+20+20 (named ep. 2), and [20]+20+20+20 (named ep. 3). The last row shows the MIOBB, namely the improvement of MEMENTO over LB. On CESNET-TLS22-80, Scratch achieves $\approx 87\%$ on 40 apps, $\approx 85\%$ on 60 apps, $\approx 84\%$ F1 Score on 80 apps.

Approach	ep. 1 F1 Score Drop [%]			ep. 2 F1 Score Drop [%]			ep. 3 F1 Score Drop [%]		
	All	Old	New	All	Old	New	All	Old	New
LB	8.73 ± 1.43	9.77 ± 3.01	7.69 ± 3.59	18.85 ± 1.62	19.22 ± 2.15	18.12 ± 3.16	24.83 ± 1.87	24.77 ± 1.81	25.01 ± 5.16
BiC	15.55 ± 10.88	15.92 ± 14.27	15.17 ± 12.37	24.08 ± 10.65	21.51 ± 10.50	29.24 ± 18.27	35.88 ± 11.42	34.77 ± 13.26	39.20 ± 21.82
MEMENTO	5.15 ± 1.37	4.98 ± 2.85	5.32 ± 2.92	8.63 ± 0.87	8.64 ± 1.74	8.59 ± 2.24	12.50 ± 0.95	12.40 ± 1.00	12.81 ± 4.27
MIOBB	+3.58	+4.79	+2.37	+10.23	+10.57	+9.53	+12.33	+12.37	+12.20

6.4. Larger set of classes: evaluation on CESNET-TLS22-80

In this section, we assess our proposal MEMENTO using an extended range of classes – leveraging the CESNET-TLS22-80 dataset – and comparing it against the leading SOTA, namely BiC, and LB. We evaluate two configurations: one involving 79+1 apps and another involving 40+40 apps, corresponding to the small-increment and big-increment scenarios, respectively. We emphasize that the MEMENTO configuration employed in 79+1 presents the parameter of 39+1 (MEMENTO_{small}) and the same is valid for 40+40 and 20+20 (MEMENTO_{big}). We use a memory of 2k samples for these experiments, as we have twice as many base classes as in the previous cases.

The results are presented in Table 6, demonstrating the *superiority of MEMENTO over SOTA approaches even when dealing with a higher number of classes*. In detail, in the small-increment scenario (79+1 apps), MEMENTO_{small} obtains improved intransigence w.r.t. BiC (+18.4% relative gain of F1 Score New) without deteriorating its robustness to forgetting. Regarding the big-increment scenario (40+40 apps), MEMENTO_{big} outperforms BiC mitigating both intransigence (+9% relative gain of F1 Score New) and forgetting issues (+6.4% relative gain of F1 Score Old) resulting in an overall improvement of $\approx 8\%$. As for Balanced Accuracy, MEMENTO_{small} obtains a 12% drop reduction on C^{new} while maintaining similar performance on the two other subsets of classes. On the other hand, MEMENTO_{big} reduces the Balanced Accuracy drop by 7.6% on C^{new} , 4.5% on C^{old} , and $\approx 6\%$ on C^{all} . Considering the LB, MEMENTO_{small} improves performance by 29.3% for C^{new} and $\approx 8\%$ for the other two subsets of classes, and MEMENTO_{big} by 5% on C^{new} , 11% on C^{old} and 8% for the overall performance.

Furthermore, we conducted a fine-grained analysis of classification errors reported in confusion matrices in Figs. 6 and 7. When examining the confusion matrices for the small-increment of 79+1 classes (resp. big-increment of 40+40 classes) shown in Fig. 6 (resp. Fig. 7), it is evident that both LB and BiC baselines display noticeable error patterns, which are considerably reduced with MEMENTO. Going into further detail, LB displays significant catastrophic forgetting, resulting in error patterns that primarily affect the old apps that are frequently confused with the new ones, with limited confusion within the latter in the big-increment scenario—see the last column in Fig. 6(a) and the top-right 40×40 squared-pattern errors in Fig. 7(a). On the other hand, BiC exhibits non-negligible catastrophic forgetting and intransigence, with old and new apps being confused within each other—see the last column and row in Fig. 6(b) and the top-right and bottom-left 40×40 squared-pattern errors in Fig. 7(b). Conversely, MEMENTO_{small} mitigates both of these issues, resulting in a practically clean confusion matrix—see the pronounced diagonal pattern in Figs. 6(c) and 7(c). Despite some performance drops still present, as indicated by points lying outside the diagonal in confusion matrices, it is noteworthy to highlight that MEMENTO achieves performance that is comparable with those of Scratch (Figs. 6(d) and 7(d)).

The results of this analysis are consistent across the devised scenarios. This analysis demonstrates how the absence of mitigation strategies (i.e., LB) clearly leads to catastrophic forgetting. Moreover, it confirms that MEMENTO mitigates BiC catastrophic forgetting and intransigence.

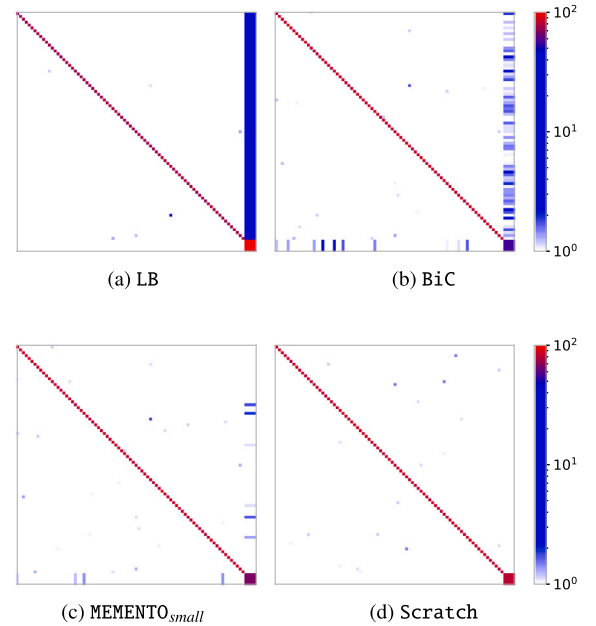


Fig. 6. Confusion matrices for baselines (i.e., LB and BiC) and MEMENTO_{small} for the 79+1 scenario. The matrices are averaged over the 40 runs. Note that the row and the column corresponding with the new app are illustrated with increased width to enhance visual representation.

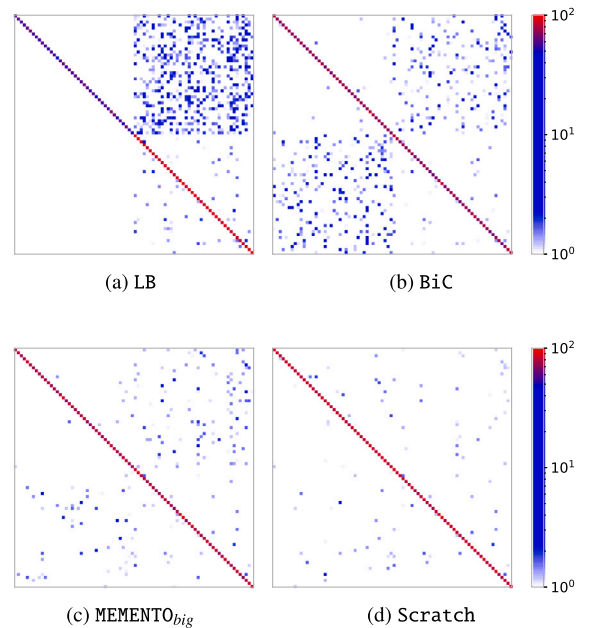


Fig. 7. Confusion matrices for baselines (i.e., LB and BiC) and MEMENTO_{big} for the 40+40 scenario. The matrices are averaged over the 20 runs.

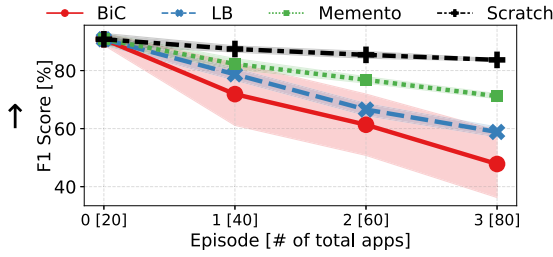


Fig. 8. F1 Score performance for C^{all} in multi-increment scenario.

6.5. Time analysis

Finally, in this section, we delve into training-time analysis to evaluate the (time) efficiency of our novel CIL approach. This analysis aims to thoroughly understand the time-related aspects of MEMENTO, highlighting potential advantages over the *training-from-scratch* approach. Accordingly, we compare MEMENTO with Scratch to demonstrate its potential to accelerate model training, and with BiC to assess the cost of performance improvement. To this aim, we adopt a *fully controlled execution environment* to measure the training time (Cfr. Section 5.1). Differently from Scratch (whose time is solely determined by the duration of each epoch), for CIL approaches we also consider the additional time required for post-training herding. Results of this analysis are reported in Table 7.

Specifically, in the big-increment scenario, $MEMENTO_{big}$ presents the same time complexity of BiC as it does not introduce any time-consuming elements but rather only variations to components already existing in BiC (cf. Sections 3.2 and 3.3). In 20+20, both $MEMENTO_{big}$ and BiC achieve $\approx 17\%$ time gain compared to Scratch on MIRAGE19 and $\approx 12\%$ on CESNET-TLS22-40. In 40+40, $MEMENTO_{big}$ maintains a notable time reduction of $\approx 12\%$ compared to Scratch.

On the other hand, in the small-increment scenario, $MEMENTO_{small}$ is mainly affected by the data augmentation that increases the number of old apps biflows (cf. Section 3.1) while the other modifications introduced do not have a significant impact (cf. Sections 3.2 and 3.3). Hence, this results in a slightly higher training time per epoch w.r.t. BiC due to the increased number of samples employed in incremental training. In 39+1 scenario, using $MEMENTO_{small}$ leads to a $>90\%$ time gain compared to Scratch across all datasets. Additionally, its training phase is slightly longer than BiC. However, this time cost is offset by $MEMENTO_{small}$ notable improvement in F1 Score on C^{new} , achieving approximately 16% for MIRAGE19 and 12% for CESNET-TLS22-40. In 79+1 scenario, $MEMENTO_{small}$ achieves an $\approx 89\%$ time gain compared to Scratch. Although its training times are slightly longer than BiC, $MEMENTO_{small}$ exhibits a significant F1 Score gain of $\approx 18\%$ for the new class. This analysis elucidates that adopting MEMENTO offers clear computational advantages in comparison to a training-from-scratch approach with up to 10x speed-up. Additionally, MEMENTO improves the TC performance of BiC at the expense of a marginal increase in computational time.

6.6. Multi-increment scenario

This ending section investigates the application of MEMENTO in a multiple increment scenario. For brevity, we present a concise analysis performed on the scenario 20+20+20+20 using CESNET-TLS22-80. The results of this analysis are shown in terms of F1 Score Drop in Table 8 and of F1 Score in Fig. 8. For this analysis, we utilize $MEMENTO_{big}$, where during each episode we add a new correction level using α to scale the logits of C^{old} (viz. the apps learned in previous episodes) and β to sum the logits of the C^{new} (viz. the apps learned in the current episode).

In detail, BiC exhibits a significant performance decay at each episode, resulting in a 48% F1 Score ($\approx 36\%$ F1 Score Drop) at the last episode for C^{all} . The baseline for the single-increment case (viz., LB)

outperforms BiC with an F1 Score (resp. F1 Score Drop) of $\approx 59\%$ (resp. $\approx 25\%$) in the last episode for all app sets. MEMENTO demonstrates clear superiority over the other two approaches, experiencing a slower decay ($\approx 5\%$) each episode and concluding with an F1 Score (resp. F1 Score Drop) of $\approx 71\%$ (resp. $\approx 13\%$) on C^{all} , C^{old} , and C^{new} . The MIOBB in the last episode, calculated considering MEMENTO and LB, is approximately 12% for C^{all} , C^{old} , and C^{new} .

7. Conclusions and future directions

In this work, we proposed a new fine-tuning CIL approach named MEMENTO. This new fine-tuning-based approach closes the performance gap between a CIL solution and the upper bound (training-from-scratch). In detail, on MIRAGE19, for multiple-app addition (i.e., 20+20 scenario), MEMENTO improves F1 Score of SOTA approaches by $\approx 3\%$ for both new and old subsets of apps, while in the case of one-app-addition (i.e., 39+1 scenario), MEMENTO provides enhanced performance by 16.3% on the new app while retaining the knowledge on existing apps. Similarly, on CESNET-TLS22-40, MEMENTO improves by 3.7% F1 Score on new apps and by 1.4% on old ones, obtaining 2.5% performance improvement overall in the 20+20 scenario, while it reaches +11.8% F1 Score on the new app in 39+1 scenario. Furthermore, MEMENTO demonstrates effectiveness when applied to a dataset with a higher number of classes (i.e., CESNET-TLS22-80). Specifically, in the 40+40 scenario, MEMENTO outperforms BiC by 9% for C^{new} , 6% for C^{old} , and achieves an overall improvement of 7%. Also, for 79+1 scenario, MEMENTO improves BiC by $\approx +18\%$ on C^{new} while remaining stable on C^{old} . Notably, in all scenarios MEMENTO provides much more stable performance, often showing half (or a fifth) of the standard deviation w.r.t. the SOTA. Moreover, MEMENTO offers a considerable advantage in terms of time compared to training models from scratch allowing to have the classifiers updated and operational on the network in a short time ($\approx 90\%$ time reduction in small-increment scenarios, $\approx 12\%$ in big-increment ones). Considering multiple subsequent increments (viz., 20+20+20+20 scenario), MEMENTO confirms its superiority over SOTA approaches: after 3 increments it limits to $\approx 13\%$ the performance loss w.r.t. the model trained from scratch, improving on the best alternatives by $\approx 12\%$.

In summary, MEMENTO constitutes a significant improvement for scenarios when a small number of new apps are added to the known ones, and it still improves on SOTA also when adding a sizable number of new apps (doubling the classes) or performing multiple subsequent increments.

Nevertheless, the gap with the ideal training-from-scratch model is not fully bridged, so there is still potential room for further improvement. Hence, as **future directions**, we plan to assess the performance of the proposed approach at varying scale, i.e., including different-sized increments (with different proportions between old and new apps) and larger evaluation with multiple incremental episodes (i.e., new app additions/old app removal). Moreover, it is of interest to evaluate our novel solution by testing it in network environments different from the training ones (i.e., with a cross-evaluation approach) or allowing our proposal to dynamically adapt to significant shifts in network traffic patterns, accommodating variations due to changes in user behavior, application updates, or external events, thus mitigating *intra-class concept-drift*. Equally important, it is valuable the investigation of hybrid approaches combining fine-tuning and model-growth families, with the objective of reducing the space and time complexity of the latter while simultaneously achieving their effectiveness. In a complementary fashion, the design and capitalization of multimodal architectures to improve CIL capabilities and performance is certainly an interesting avenue of future research. Last but not least, it is worth exploring “exemplar-free” solutions with the objective of privacy preservation and the easier dissemination of pre-trained models (e.g., with the intent of fostering customization of traffic classifiers in small office home office contexts).

CRedit authorship contribution statement

Francesco Cerasuolo: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation. **Alfredo Nascita:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis. **Giampaolo Bovenzi:** Writing – review & editing, Writing – original draft, Visualization, Software, Resources, Methodology. **Giuseppe Aceto:** Writing – review & editing, Supervision, Project administration, Conceptualization. **Domenico Ciunzo:** Writing – review & editing, Supervision, Methodology, Conceptualization. **Antonio Pescapè:** Writing – review & editing, Supervision, Project administration, Conceptualization. **Dario Rossi:** Writing – review & editing, Project administration, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgment

This work has been funded in the framework of the Huawei Innovation Lab project on “Network Traffic and AI enabled Network Technologies” by Huawei Technologies France SASU at DIETI, University of Napoli Federico II.

References

- [1] Alberto Dainotti, Antonio Pescapè, Kimberly C. Claffy, Issues and future directions in traffic classification, *IEEE Netw.* 26 (1) (2012) 35–40.
- [2] Fannia Pacheco, Ernesto Exposito, Mathieu Gineste, Cedric Baudoin, Jose Aguilar, Towards the deployment of machine learning solutions in network traffic classification: a systematic survey, *IEEE Commun. Surv. Tutorials* 21 (2) (2018) 1988–2014.
- [3] Ericsson, Ericsson mobility report, 2023, <https://www.ericsson.com/49dd9d/assets/local/reports-papers/mobility-report/documents/2023/ericsson-mobility-report-june-2023.pdf>. Accessed: 2024-04-05.
- [4] Zhanyi Wang, The applications of deep learning on traffic identification, *BlackHat USA* 24 (11) (2015) 1–10.
- [5] Wei Wang, Ming Zhu, Jinlin Wang, Xuewen Zeng, Zhongzhen Yang, End-to-end encrypted traffic classification with one-dimensional convolution neural networks, in: *Proc. IEEE Int. Conference Intelligence and Security Informatics, ISI*, 2017, pp. 43–48.
- [6] Giuseppe Aceto, Domenico Ciunzo, Antonio Montieri, Antonio Pescapè, Mobile encrypted traffic classification using deep learning: Experimental evaluation, lessons learned, and challenges, *IEEE Trans. Netw. Serv. Manag.* 16 (2) (2019) 445–458.
- [7] Manuel Lopez-Martin, Belen Carro, Antonio Sanchez-Esguevillas, Jaime Lloret, Network traffic classifier with convolutional and recurrent neural networks for internet of things, *IEEE Access* 5 (2017) 18042–18050.
- [8] Giuseppe Aceto, Domenico Ciunzo, Antonio Montieri, Valerio Persico, Antonio Pescapè, MIRAGE: mobile-app traffic capture and ground-truth creation, in: *Proc. IEEE Int. Conference on Computing Communication Security, ICCS*, 2019, pp. 1–8.
- [9] Mohammad Lotfollahi, Mahdi Jafari Siavoshani, Ramin Shirali Hossein Zade, Mohammadsadegh Saberian, Deep packet: A novel approach for encrypted traffic classification using deep learning, *Springer Soft Comput.* 24 (3) (2020).
- [10] Number of new Android app releases via Google Play per month, 2023, <http://web.archive.org/web/20230329105804/https://www.statista.com/statistics/1020956/android-app-releases-worldwide/>. Accessed: 2024-04-05.
- [11] Jielun Zhang, Fuhao Li, Feng Ye, Hongyu Wu, Autonomous unknown-application filtering and labeling for DL-based traffic classifier update, in: *Proc. IEEE Conference Computer Communications Workshops, INFOCOM WKSHPS*, 2020, pp. 397–405.
- [12] Yafei Sang, Mao Tian, Yongzheng Zhang, Peng Chang, Shuyuan Zhao, In-creAIBMF: incremental learning for encrypted mobile application identification, in: *20th Int. Conference on Algorithms and Architectures for Parallel Processing, ICA3PP*, 2020, pp. 494–508.
- [13] Yige Chen, Tianning Zang, Yongzheng Zhang, Yuan Zhou, Linshu Ouyang, Peng Yang, Incremental learning for mobile encrypted traffic classification, in: *Proc. IEEE Int. Conference on Communications, ICC*, 2021, pp. 1–6.
- [14] Giampaolo Bovenzi, Lixuan Yang, Alessandro Finamore, Giuseppe Aceto, Domenico Ciunzo, Antonio Pescapè, Dario Rossi, A first look at class incremental learning in deep learning mobile traffic classification, in: *Proc. IFIP Traffic Monitoring Analysis, TMA*, 2021.
- [15] Zheng Wu, Yu-ning Dong, Xiaohui Qiu, Jiong Jin, Online multimedia traffic classification from the QoS perspective using deep learning, *Elsevier Comput. Netw.* 204 (2022) 108716.
- [16] Zhuoxue Song, Ziming Zhao, Fan Zhang, Gang Xiong, Guang Cheng, Xinjie Zhao, Shize Guo, Binbin Chen, I2-RNN: An incremental and interpretable recurrent neural network for encrypted traffic classification, *IEEE Trans. Depend. Secure Comput.* (2023).
- [17] Xiuli Ma, Wenbin Zhu, Jieliang Wei, Yanliang Jin, Dongsheng Gu, Rui Wang, EETC: An extended encrypted traffic classification algorithm based on variant resnet network, *Elsevier Comput. Secur.* 128 (2023) 103175.
- [18] Wenbin Zhu, Xiuli Ma, Yanliang Jin, Rui Wang, ILET: incremental learning for encrypted traffic classification using generative replay and exemplar, *Elsevier Comput. Netw.* 224 (2023) 109602.
- [19] Wei Cai, Chengshang Hou, Chang Liu, Gaopeng Gou, Gang Xiong, Zhen Li, PTC: Prompt-based continual encrypted traffic classification, in: *2023 26th Int. Conference on Computer Supported Cooperative Work in Design, CSCWD, IEEE*, 2023, pp. 1557–1562.
- [20] Giampaolo Bovenzi, Alfredo Nascita, Lixuan Yang, Alessandro Finamore, Giuseppe Aceto, Domenico Ciunzo, Antonio Pescapè, Dario Rossi, Benchmarking class incremental learning in deep learning traffic classification, *IEEE Trans. Netw. Serv. Manag.* (2023).
- [21] Marc Masana, Xialei Liu, Bartłomiej Twardowski, Mikel Menta, Andrew D Bagdanov, Joost van de Weijer, Class-incremental learning: survey and performance evaluation on image classification, *IEEE Trans. Pattern Anal. Mach. Intell.* 45 (5) (2022) 5513–5533.
- [22] Jan Luxemburk, Tomáš Čejka, Fine-grained TLS services classification with reject option, *Elsevier Comput. Netw.* (ISSN: 1389-1286) 220 (2023) 109467.
- [23] Ian J Goodfellow, Mehdi Mirza, Da Xiao, Aaron Courville, Yoshua Bengio, An empirical investigation of catastrophic forgetting in gradient-based neural networks, 2013, arXiv preprint arXiv:1312.6211.
- [24] Tyler L. Hayes, Kushal Kafle, Robik Shrestha, Manoj Acharya, Christopher Kanan, REMIND your neural network to prevent catastrophic forgetting, in: *Proc. European Conference Computer Vision, ECCV*, 2020.
- [25] Ronald Kemker, Christopher Kanan, FearNet: brain-inspired model for incremental learning, in: *Proc. IEEE Int. Conference Learning Representation, ICLR*, 2018.
- [26] Wei Wang, Ming Zhu, Jinlin Wang, Xuewen Zeng, Zhongzhen Yang, End-to-end encrypted traffic classification with one-dimensional convolution neural networks, in: *IEEE ISI*, 2017, pp. 43–48.
- [27] Chang Liu, Longtao He, Gang Xiong, Zigang Cao, Zhen Li, FS-Net: A flow sequence network for encrypted traffic classification, in: *IEEE INFOCOM*, 2019, pp. 1171–1179.
- [28] Zhuang Zou, Jingguo Ge, Hongbo Zheng, Yulei Wu, Chunjing Han, Zhongjiang Yao, Encrypted traffic classification with a convolutional long short-term memory neural network, in: *IEEE HPCC/SmartCity/DSS*, 2018, pp. 329–334.
- [29] Alfredo Nascita, Antonio Montieri, Giuseppe Aceto, Domenico Ciunzo, Valerio Persico, Antonio Pescapè, XAI meets mobile traffic classification: Understanding and improving multimodal deep learning architectures, *IEEE Trans. Netw. Serv. Manag.* 18 (4) (2021) 4225–4246.
- [30] Iman Akbari, Mohammad A Salahuddin, Leni Ven, Noura Limam, Raouf Boutaba, Bertrand Mathieu, Stephanie Moteau, Stephane Tuffin, A look behind the curtain: Traffic classification in an increasingly encrypted web, *Proc. ACM Meas. Anal. Comput. Syst.* 5 (1) (2021) 1–26.
- [31] Xinjie Lin, Gang Xiong, Gaopeng Gou, Zhen Li, Junzheng Shi, Jing Yu, ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification, in: *Proc. of the ACM Web Conference*, 2022, pp. 633–642.
- [32] Geoffrey Hinton, Oriol Vinyals, Jeff Dean, Distilling the knowledge in a neural network, in: *Proc. NIPS Deep Learning Representation Learning Workshop*, 2015.
- [33] Yue Wu, Yinpeng Chen, Lijuan Wang, Yuancheng Ye, Zicheng Liu, Yandong Guo, Yun Fu, Large scale incremental learning, in: *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR*, 2019, pp. 374–382.
- [34] Bo Zhao, Konda Reddy Mopuri, Hakan Bilen, Dataset condensation with gradient matching, in: *Proc. IEEE Int. Conference Learning Representation, ICLR*, 2021.

- [35] Da-Wei Zhou, Qi-Wei Wang, Zhi-Hong Qi, Han-Jia Ye, De-Chuan Zhan, Ziwei Liu, Deep class-incremental learning: A survey, 2023, arXiv preprint arXiv: 2302.03648.
- [36] Zifeng Wang, Zizhao Zhang, Chen-Yu Lee, Han Zhang, Ruoxi Sun, Xiaoqi Ren, Guolong Su, Vincent Perot, Jennifer Dy, Tomas Pfister, Learning to prompt for continual learning, in: Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR, 2022, pp. 139–149.
- [37] Gerard Draper-Gil, Arash Habibi Lashkari, Mohammad Saiful Islam Mamun, Ali A Ghorbani, Characterization of encrypted and VPN traffic using time-related, in: Proc. of the 2nd Int. Conference on Information Systems Security and Privacy, ICISSP, 2016, pp. 407–414.
- [38] Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, Georg Sperl, Christoph H Lampert, iCARL: incremental classifier and representation learning, in: Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR, 2017, pp. 2001–2010.
- [39] Arslan Chaudhry, Puneet K Dokania, Thalaiyasingam Ajanthan, Philip HS Torr, Riemannian walk for incremental learning: Understanding forgetting and intransigence, in: Proc. European Conference Computer Vision, ECCV, 2018, pp. 532–547.
- [40] Jihwan Bang, Heesu Kim, YoungJoon Yoo, Jung-Woo Ha, Jonghyun Choi, Rainbow memory: Continual learning with a memory of diverse samples, in: Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR, 2021, pp. 8218–8227.
- [41] Minsoo Kang, Jaeyoo Park, Bohyung Han, Class-incremental learning by knowledge distillation with adaptive feature consolidation, in: Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR, 2022, pp. 16071–16080.
- [42] Pietro Buzzega, Matteo Boschini, Angelo Porrello, Davide Abati, Simone Calderara, Dark experience for general continual learning: a strong, simple baseline, Adv. Neural Inf. Process. Syst. 33 (2020) 15920–15930.
- [43] Lutz Prechelt, Early stopping-but when? in: Neural Networks: Tricks of the Trade, Springer, 2002, pp. 55–69.
- [44] Bowen Zhao, Xi Xiao, Guojun Gan, Bin Zhang, Shu-Tao Xia, Maintaining discrimination and fairness in class incremental learning, in: Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR, 2020, pp. 13208–13217.
- [45] Eden Belouadah, Adrian Popescu, ScaIL: classifier weights scaling for class incremental learning, in: Proc. IEEE/CVF Winter Conference on Applications of Computer Vision, WACV, 2020, pp. 1266–1275.
- [46] Giuseppe Aceto, Domenico Ciunzio, Antonio Montieri, Antonio Pescapè, MIMETIC: mobile encrypted traffic classification using multimodal deep learning, Elsevier Comput. Netw. 165 (2019) 106944.
- [47] Lixuan Yang, Alessandro Finamore, Feng Jun, Dario Rossi, Deep learning and zero-day traffic classification: Lessons learned from a commercial-grade dataset, IEEE Trans. Netw. Serv. Manag. 18 (4) (2021) 4103–4118.
- [48] Francesco Cerasuolo, Giampaolo Bovenzi, Christian Marescalco, Francesco Cirillo, Domenico Ciunzio, Antonio Pescapè, Adaptive intrusion detection systems: class incremental learning for iot emerging threats, in: 2023 IEEE International Conference on Big Data (BigData), IEEE, 2023, pp. 3547–3555.



Francesco Cerasuolo is a Ph.D. student at the Department of Electrical Engineering and Information Technology of the University of Napoli Federico II. He received his M.S. Laurea Degree in Computer Engineering in July 2022 from the same University. His research interests include traffic classification, machine and deep learning, and class incremental learning.



Alfredo Nascita is a Ph.D. Student in Information Technology and Electrical Engineering at DIETI, University of Napoli Federico II. He received his M.S. Laurea Degree in Computer Engineering from the same University in March 2021. His research interests include traffic classification, machine and deep learning, and explainable artificial intelligence.



Giampaolo Bovenzi is a Assistant Professor at DIETI, University of Napoli Federico II, since October 2023. He received his Ph.D. degree at the same University in June 2022. His research interests focus on (anonymized and encrypted) traffic classification, network security (with a focus on IoT), and blockchain. He has co-authored 15 papers in international journals and conference proceedings.



Giuseppe Aceto is an Associate Professor at the University of Napoli Federico II. He has a Ph.D. in telecommunication engineering from the same University. His work falls in monitoring of network performance and security (focusing on censorship) both in traditional and SDN network environments. He is also working on bioinformatics and ICTs applied to health. He is the recipient of the best paper awards at IEEE ISCC 2010 and IEEE ICCCS 2019, the Computer Networks 2020 Best Paper Award, and the 2018 Best Journal Paper Award by IEEE CSIM.



Domenico Ciunzio is a Tenure-Track Professor at University of Napoli Federico II. He holds a Ph.D. from the University of Campania L. Vanvitelli. He is the recipient of two Paper awards (IEEE ICCCS 2019 and Elsevier ComNet 2020), the 2019 IEEE AESS Exceptional Service award, the 2020 IEEE SENSORS COUNCIL Early-Career Technical Achievement award and the 2021 IEEE AESS Early-Career Award. His research interests are data fusion, network analytics, IoT, and AI.



Antonio Pescapè is a Full Professor of computer engineering at the University of Napoli "Federico II". His work focuses on Internet technologies and more precisely on measurement, monitoring, and analysis of the Internet. Recently, he is working on bioinformatic and ICTs for a smarter health. Antonio has co-authored more than 200 conference and journal papers and is the recipient of a number of research awards. Also, he has served as an independent reviewer/evaluator of research projects/project proposals co-funded by several governments and agencies.



Dario Rossi is a Network AI CTO and the Director of the DataCom Lab, Huawei Technologies, France. Before joining Huawei in 2018, he held a Full Professor positions with Telecom Paris and Ecole Polytechnique and was a holder of 'Cisco's Chair NewNet Paris. He has coauthored 15 patents and over 200 papers in leading conferences and journals, that received nine best paper awards, a Google Faculty Research Award in 2015 and an IRTF Applied Network Research Prize in 2016. He is a Senior Member of ACM.